

Robot Perception and Learning

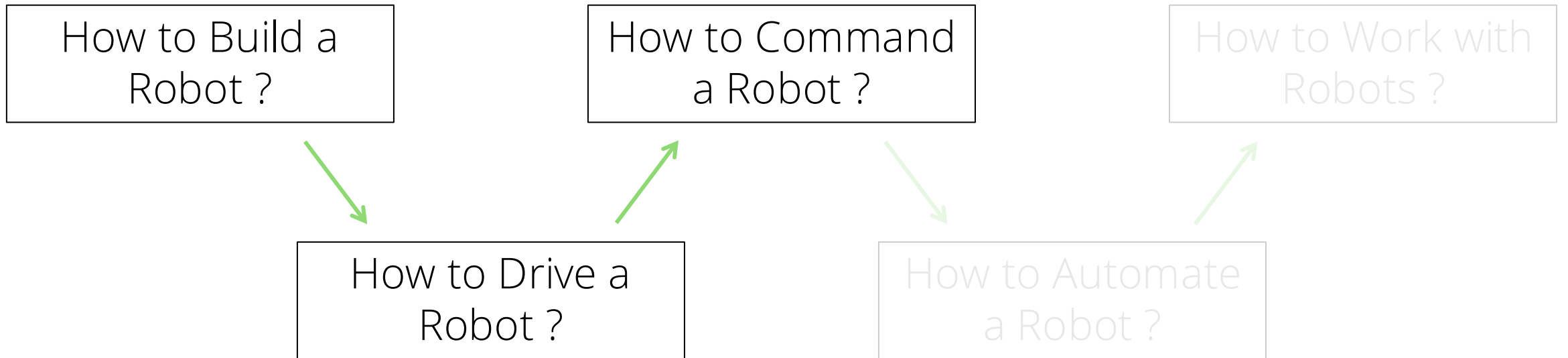
Feedback Control

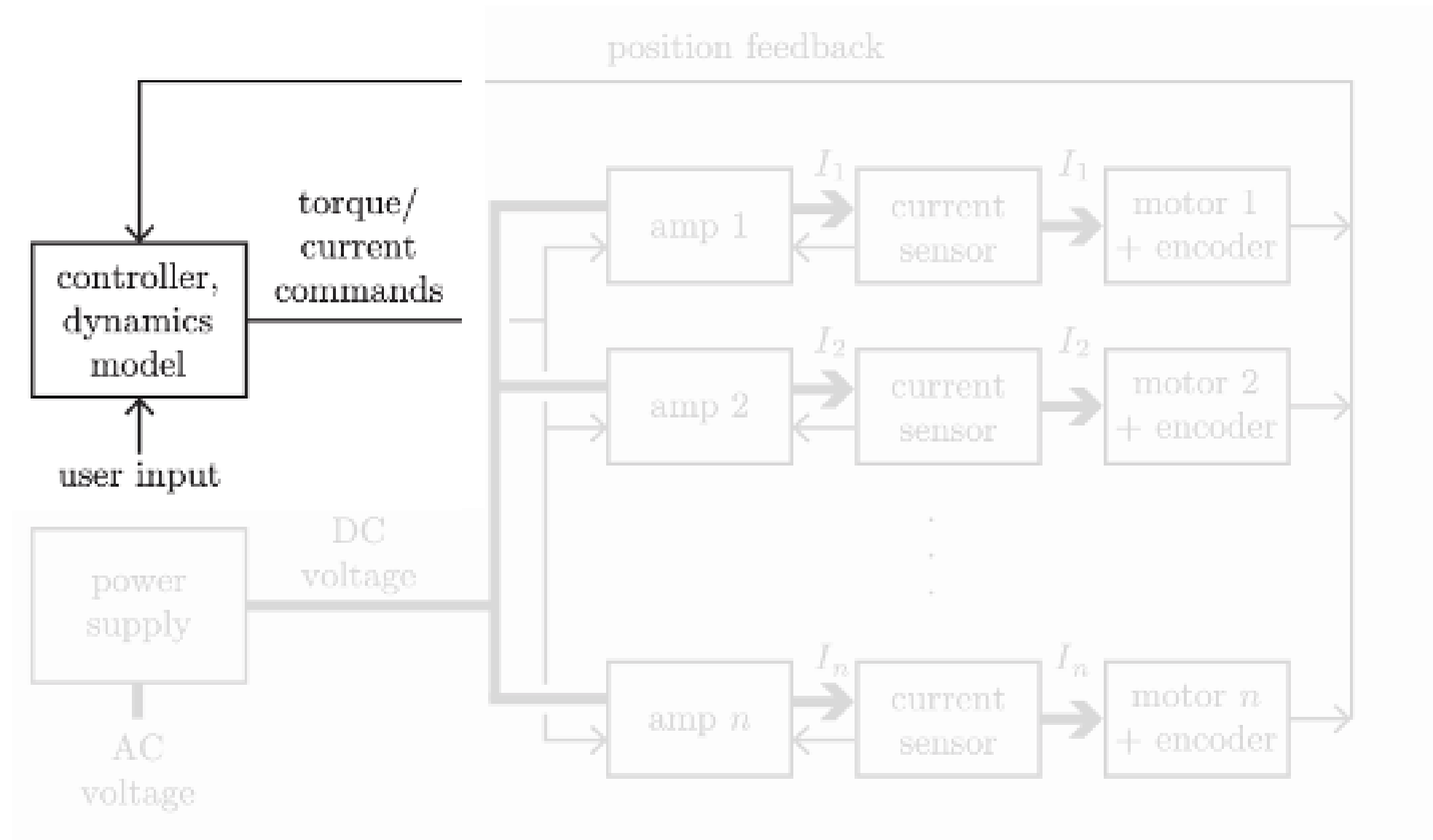
Tsung-Wei Ke

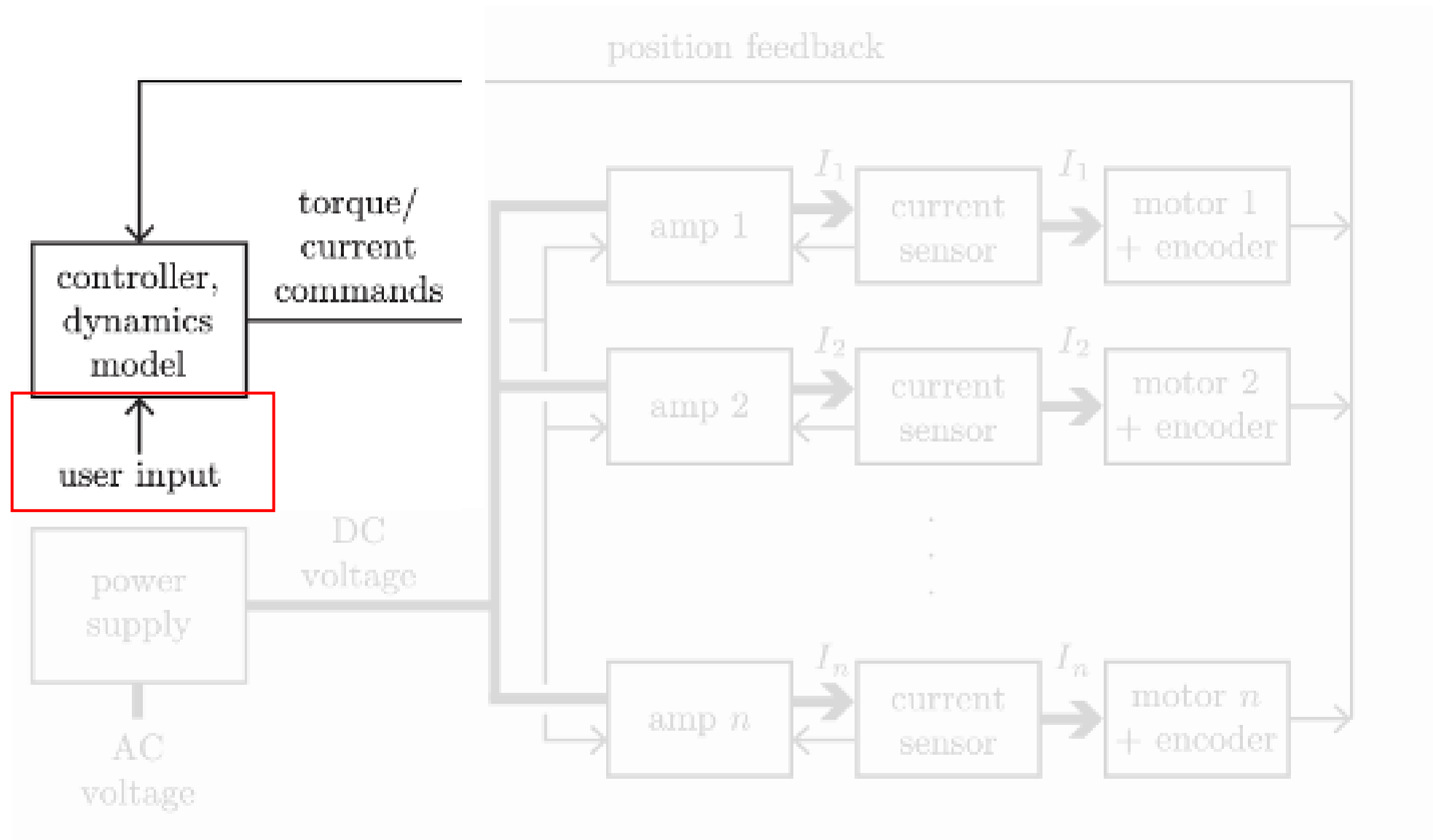
Fall 2026



Recap: Robotics is a System



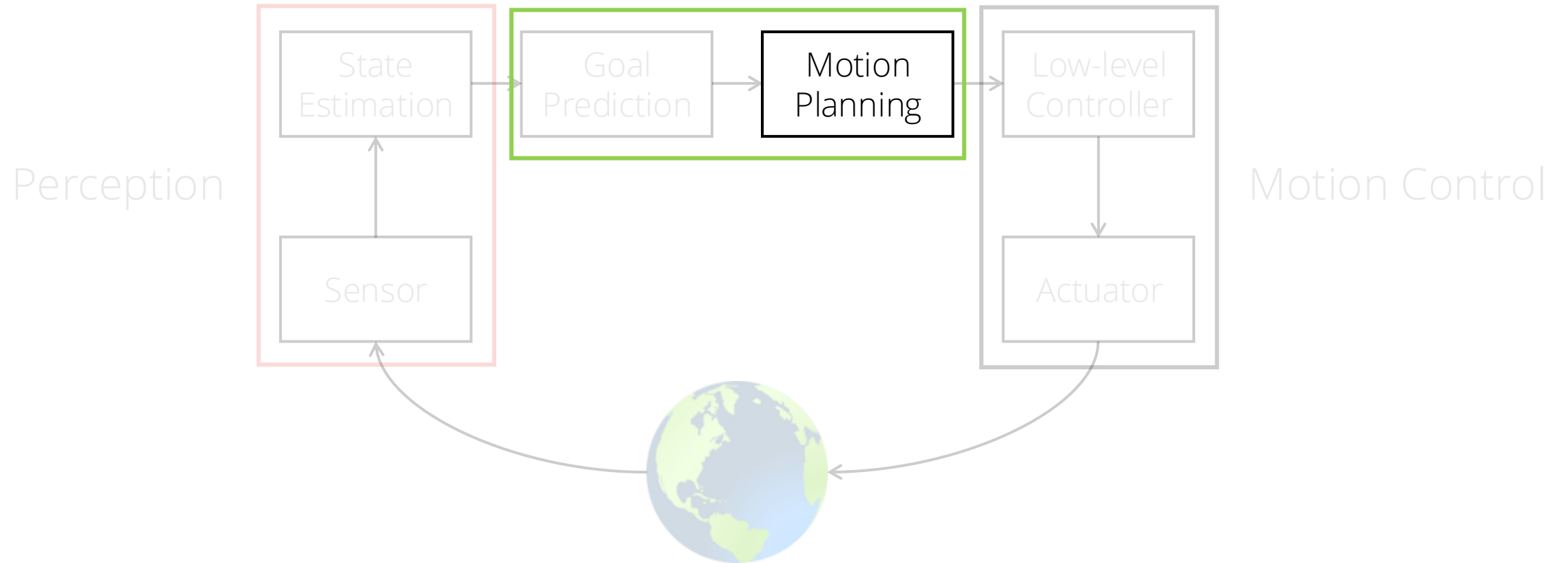




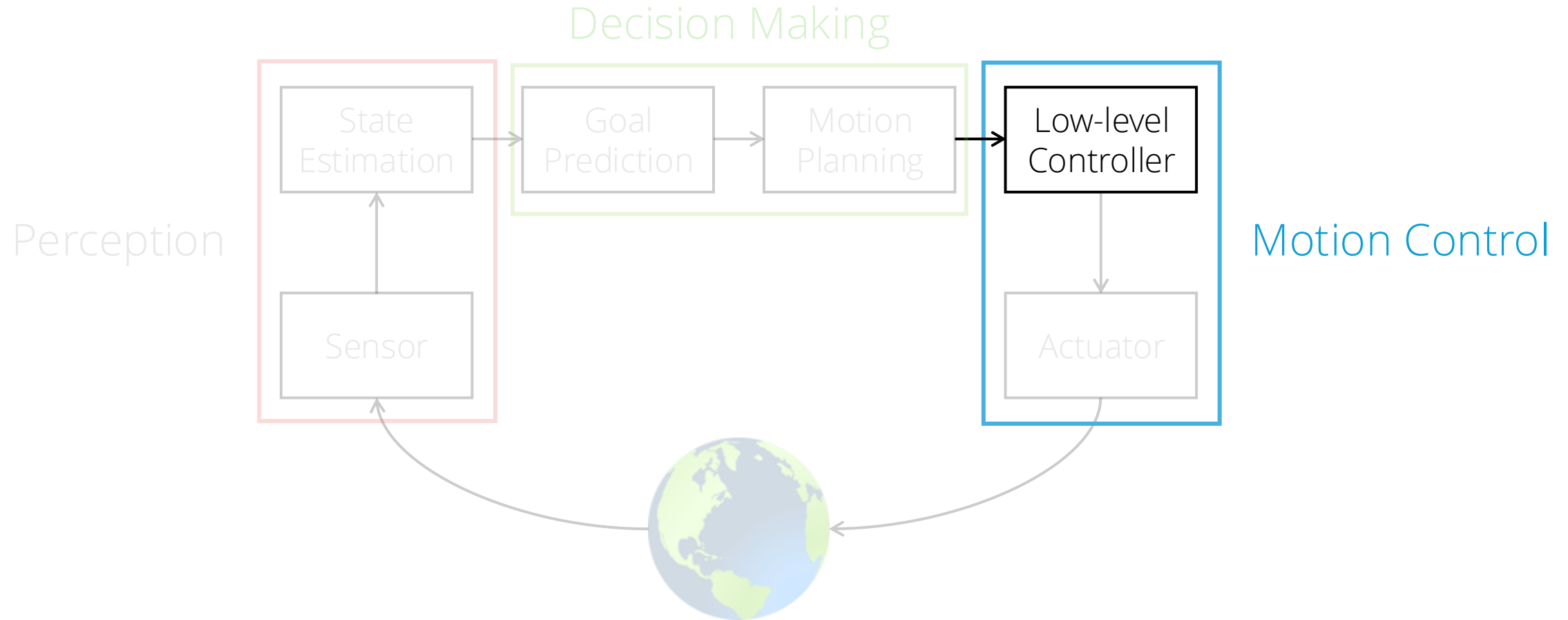
Recap: Motion Planning

We will revisit decision making

Decision Making



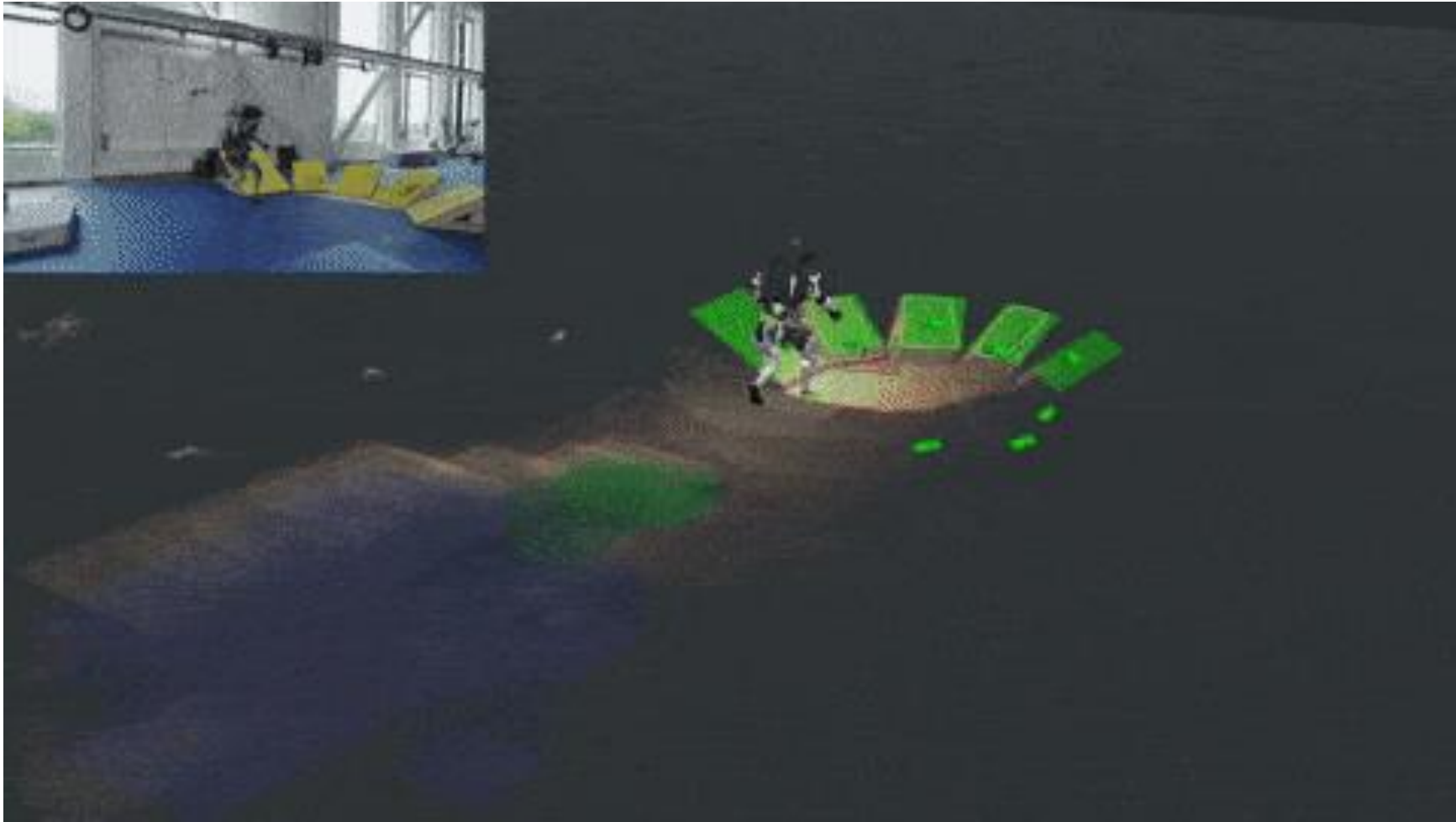
Today's Class Focuses on Controller



Content

- Open-loop control vs. close-loop control
- PID controllers
- Stability analysis of PID controllers
- Control in robotic manipulation
 - The effect of PD gain
 - Joint-space control vs. task-space control
 - Position control vs. force control
 - Indirect force control (impedance control)

How to Control Robots to Follow Plans?



Control is Hard, as Robots may be Underactuated...

Underactuated system: a mechanical system that cannot be commanded to follow arbitrary trajectories in configuration space. Obvious cases are systems that have less actuators than degrees of freedom.

Definition 1.1 (Underactuated Control Differential Equations) A second-order control differential equation described by the equations

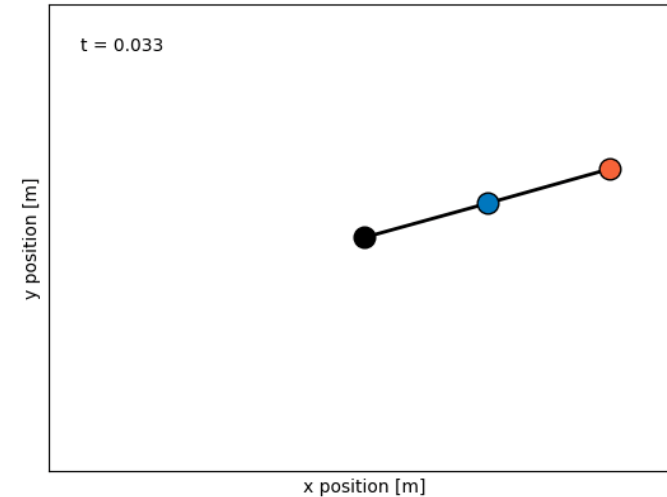
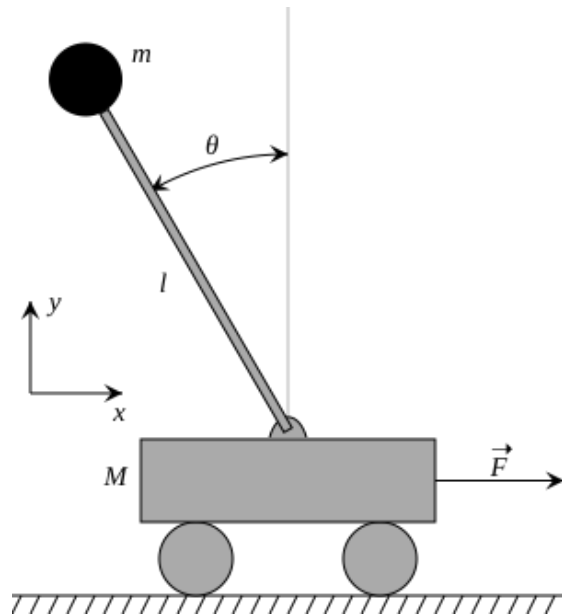
$$\ddot{\mathbf{q}} = \mathbf{f}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{u}, t) \quad (1)$$

is *fully actuated* in state $\mathbf{x} = (\mathbf{q}, \dot{\mathbf{q}})$ and time t if the resulting map $\mathbf{f}$ is surjective: for every $\ddot{\mathbf{q}}$ there exists a $\mathbf{u}$ which produces the **desired response**. Otherwise it is *underactuated* (at state $\mathbf{x}$ at time t).

$\mathbf{q}$: a vector of robot configurations; $\dot{\mathbf{q}}$: a vector of velocities

Control is Hard, as Robots may be Underactuated...

Underactuated system: a mechanical system that cannot be commanded to follow arbitrary trajectories in configuration space. Obvious cases are systems that have less actuators than degrees of freedom.

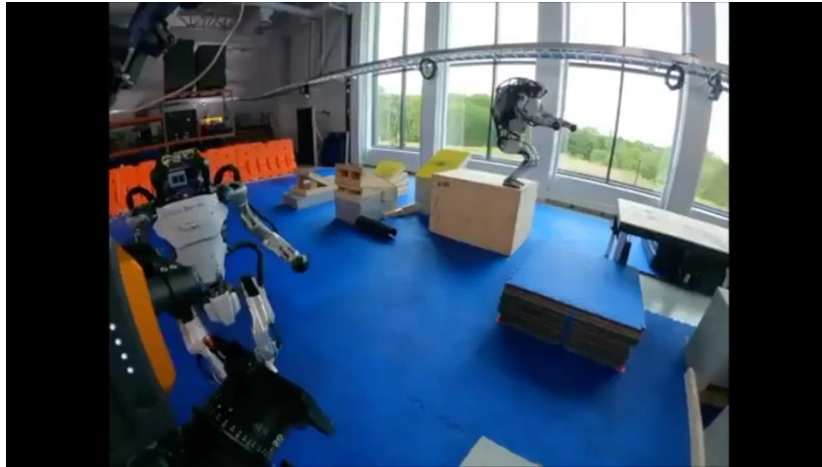


Video credit W. Felix, K. Shivesh, S. Lasse J., V. Shubam and J. Ma.

We Ignore the Dynamics...



<https://www.youtube.com/watch?v=RqajKat0v-4>



https://www.youtube.com/watch?v=-9EM5_VFlt8&t=2s

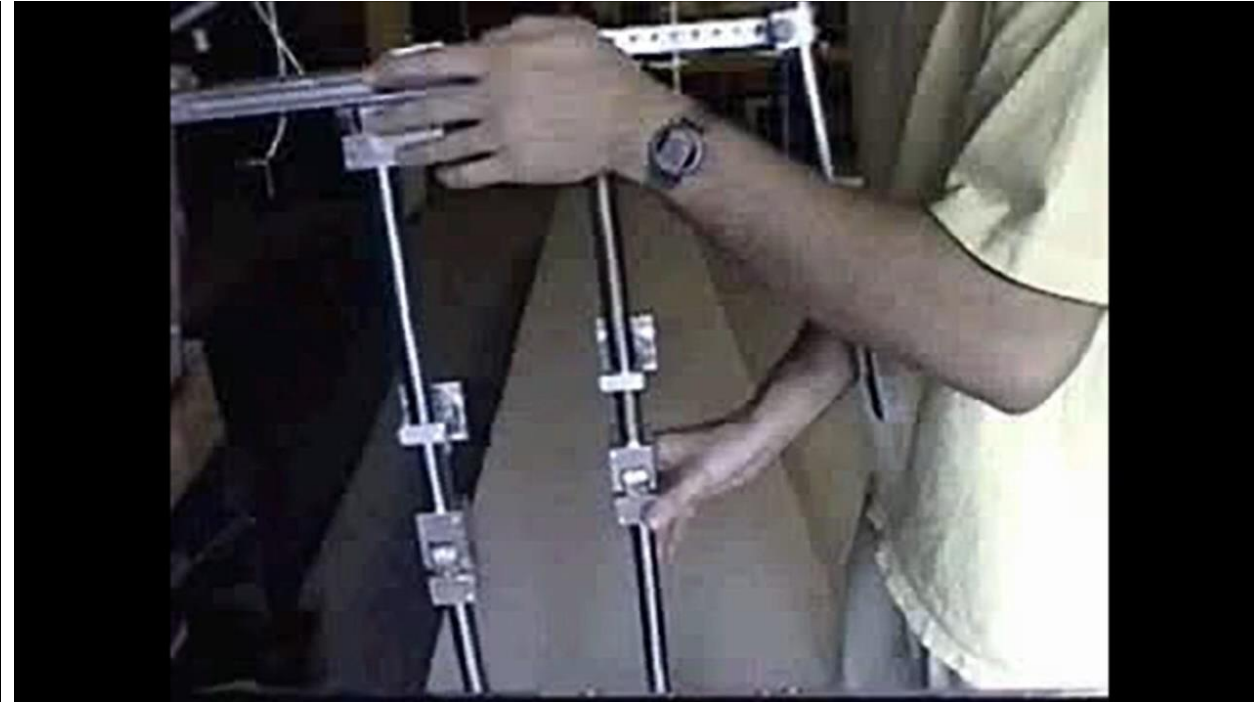


<https://www.youtube.com/watch?v=eFsT3Qglvol>

Control Systems that Cancel vs. Involve Dynamic



Honda Asimo: Video credit <https://youtu.be/vA0xLVCb-OA?si=Z4MKkuM4Wr0H7NJw>



Steven Collins. Video credit <https://youtu.be/PK7cgLJD2nQ?si=5d-Ke74URVlejBpX>

Which one looks more natural? Which one consumes less energy?

Content

- Open-loop control vs. close-loop control
- PID controllers
- Stability analysis of PID controllers
- Control in robotic manipulation
 - The effect of PD gain
 - Position control vs. force control
 - Joint-space control vs. task-space control
 - Indirect force control (impedance control)

Two Types of Control Strategy

Open-loop Control

Torque $\tau_1, \dots, \tau_T$



Close-loop Control

Torque τ_t



State s_{t+1}

Two Types of Control Strategy

Open-loop Control

Torque $\tau_1, \dots, \tau_T$



Close-loop Control

Torque τ_t



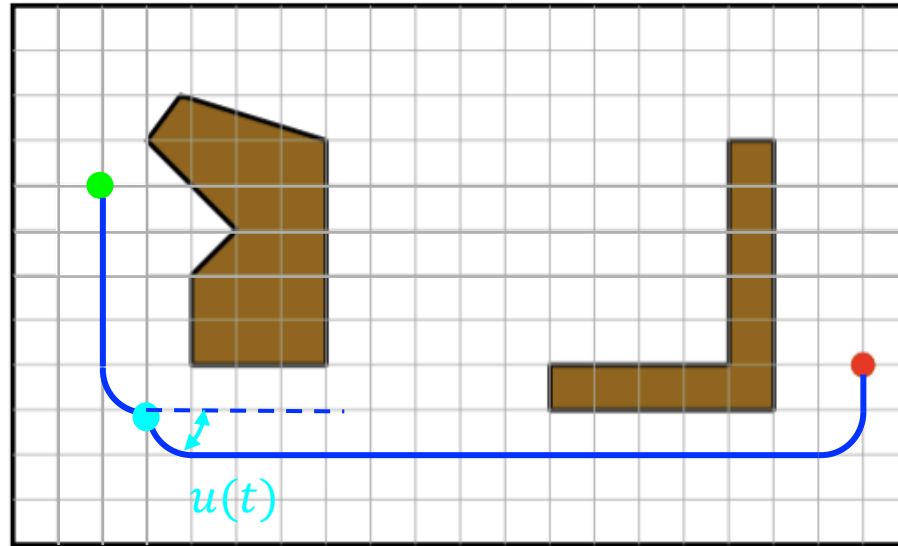
State s_{t+1}



Why not Extending Motion Planning to Generating Control Trajectory?

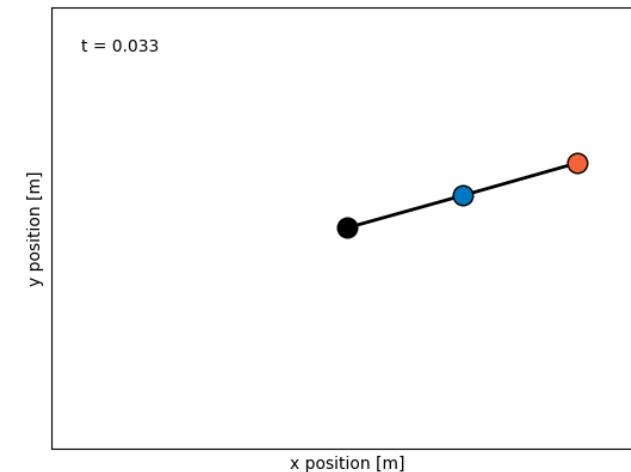
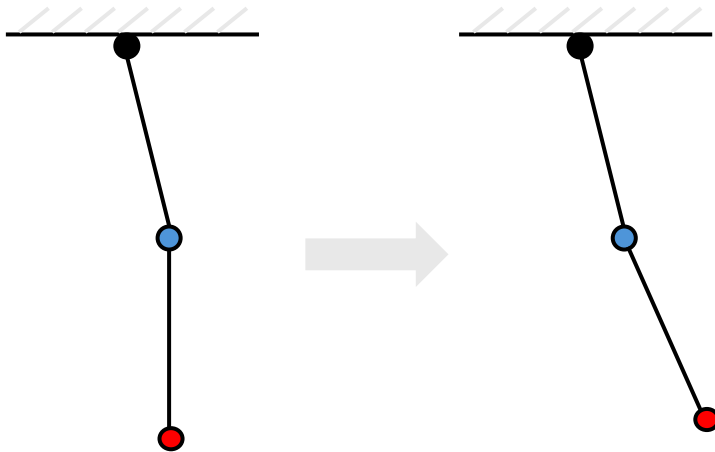
Control trajectory $u(t)$:

- Acceleration, determined by imposed force/torque, at time t
- Velocity at time t (e.g. stepper motors)



Control is Hard, as Robots may be Underactuated...

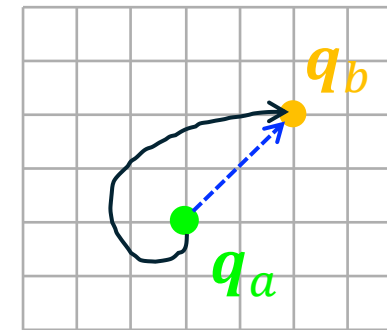
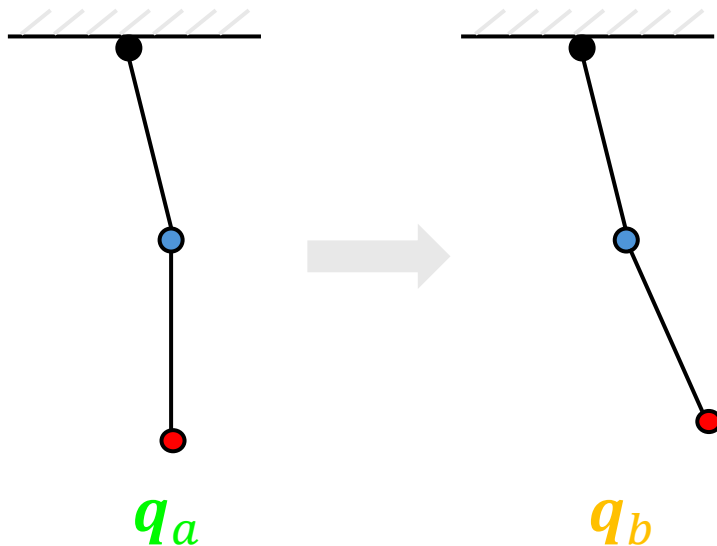
- **Underactuated system:** a mechanical system that cannot be commanded to follow arbitrary trajectories in configuration space. Obvious cases are systems that have less actuators than degrees of freedom.
- The double pendulum with one missing actuator is underactuated. The neighboring configurations take more motions to achieve



Video credit W. Felix, K. Shivesh, S. Lasse J., V. Shubam and J. Ma.

Control is Hard, as Robots may be Underactuated...

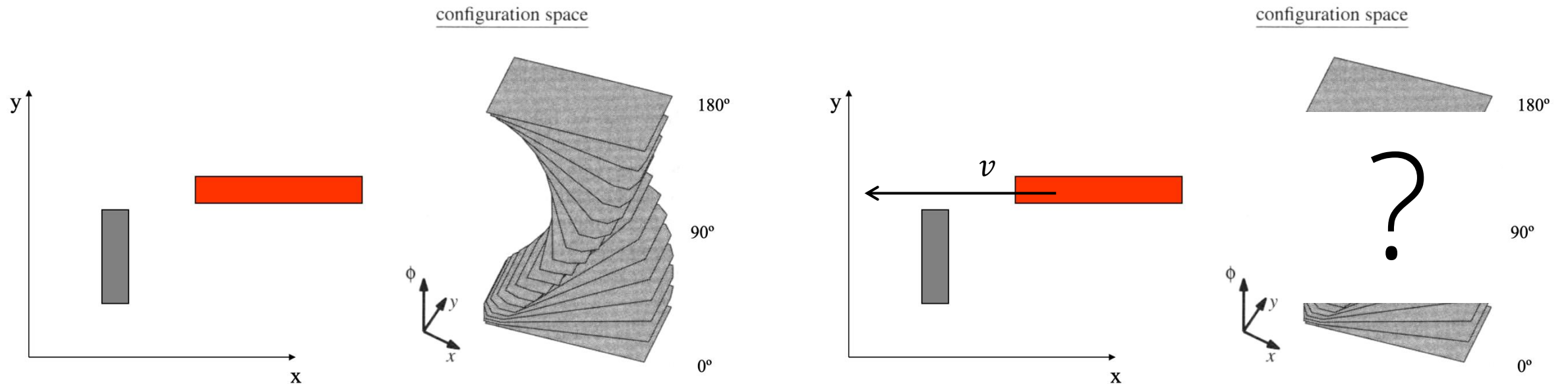
- **Underactuated system:** a mechanical system that cannot be commanded to follow arbitrary trajectories in configuration space. Obvious cases are systems that have less actuators than degrees of freedom.
- The double pendulum with one missing actuator is underactuated. The neighboring configurations take more motions to achieve
- Idea: Building graph by sampling real/simulated interactions



Actual trajectory depends on force constraints and velocity conditions

Can We Generate Control Trajectories with Motion Planning Methods?

- To generate control trajectories, we need to take dynamic into consideration
- Previously, we only consider “configurations” (joint angles) of a robot. Such representations ignore “dynamics” of a robot.
 - Is the $\mathcal{C}$ -space obstacle fixed when the initial velocity $\mathbf{v}$ of an object varies?



State-space Representations

- Generalize the planning space from configurations (the configuration space $\mathcal{C}$) to dynamics (the state space $\mathcal{X}$)

- **Definition:** a state vector $x \in \mathcal{X}$ as $x = \begin{bmatrix} q \\ \dot{q} \\ \vdots \end{bmatrix}$, then we have $\dot{x} = \begin{bmatrix} \dot{q} \\ \ddot{q} \\ \vdots \end{bmatrix} = f(x, u, t)$,

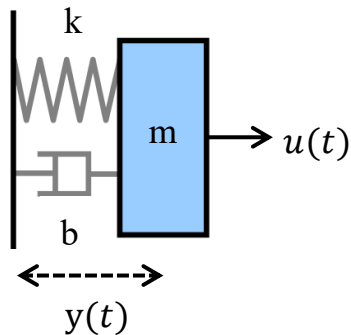
where $q \in \mathcal{C}$, u denotes controls (e.g. torque) and function f describes how the motion changes conditioned on the state x , control u and time t

State-space Representations

- Generalize the planning space from configurations (the configuration space $\mathcal{C}$) to dynamics (the state space $\mathcal{X}$)

- Definition:** a state vector $x \in \mathcal{X}$ as $x = \begin{bmatrix} q \\ \dot{q} \\ \vdots \end{bmatrix}$, then we have $\dot{x} = \begin{bmatrix} \dot{q} \\ \ddot{q} \\ \vdots \end{bmatrix} = f(x, u, t)$,

where $q \in \mathcal{C}$, u denotes controls (e.g. torque) and function f describes how the motion changes conditioned on the state x , control u and time t



A 2nd-order equation of motion:

$$m\ddot{q}(t) = u(t) - b\dot{q}(t) - kq(t)$$

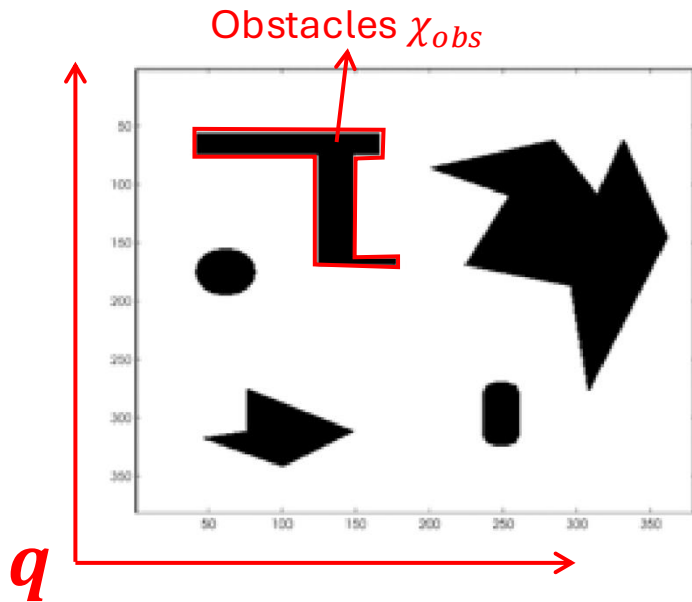
Rewrite the equation as:

$$\dot{x} = \begin{bmatrix} \dot{q}(t) \\ \ddot{q}(t) \end{bmatrix} = \begin{bmatrix} \dot{q}(t) \\ \frac{1}{m}u(t) - \frac{b}{m}\dot{q}(t) - \frac{k}{m}q(t) \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{b}{m} \end{bmatrix}}_A x(t) + \underbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}}_B u(t)$$

Obstacles in the State Space

Let's assume we have a point mass robot

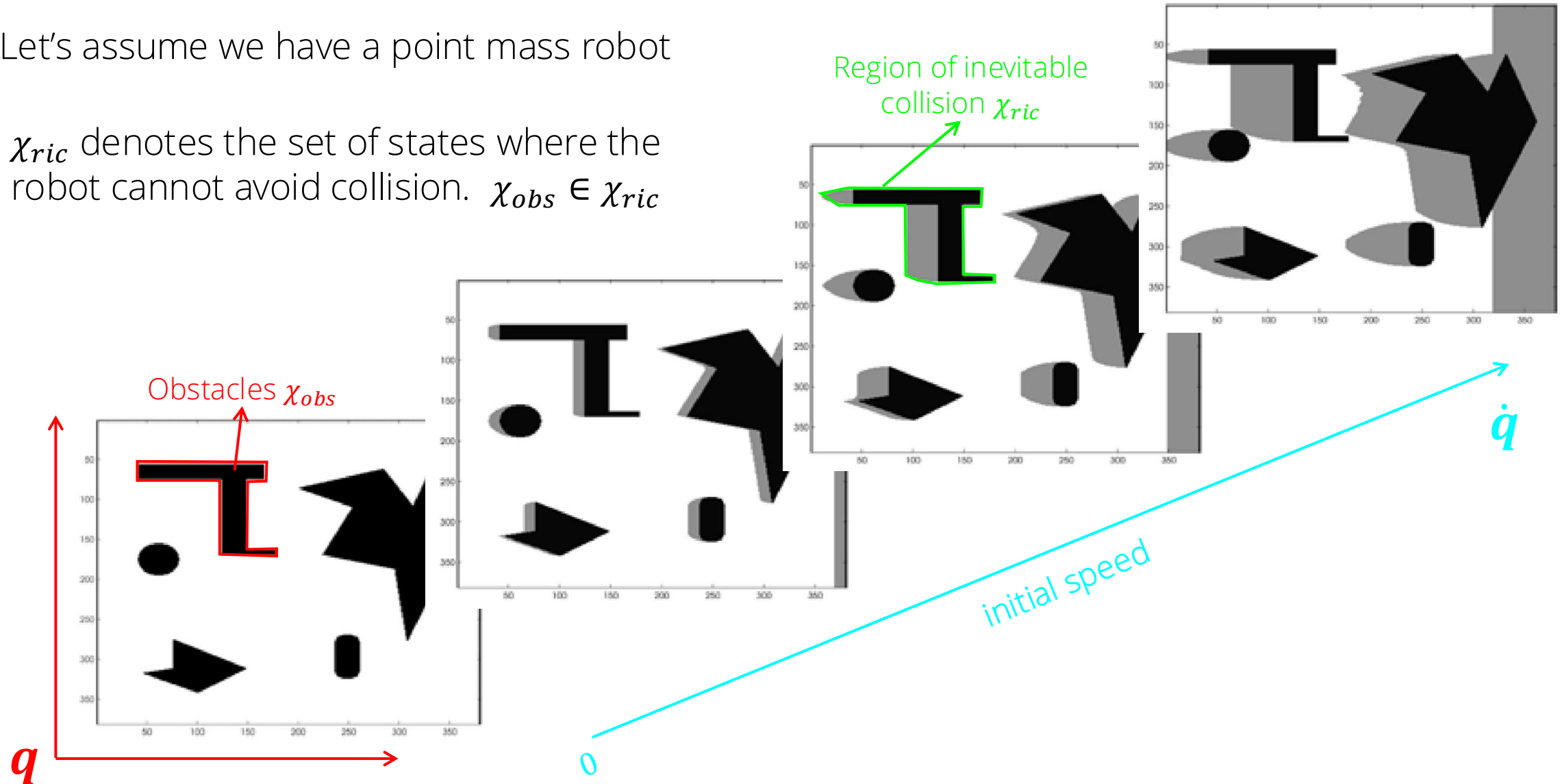
χ_{ric} denotes the set of states where the robot cannot avoid collision. $\chi_{obs} \in \chi_{ric}$



Obstacles in the State Space

Let's assume we have a point mass robot

χ_{ric} denotes the set of states where the robot cannot avoid collision. $\chi_{obs} \in \chi_{ric}$



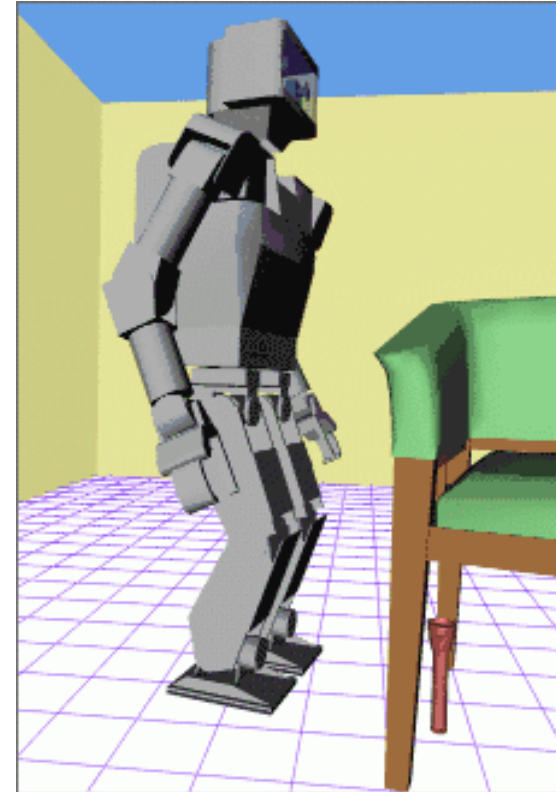
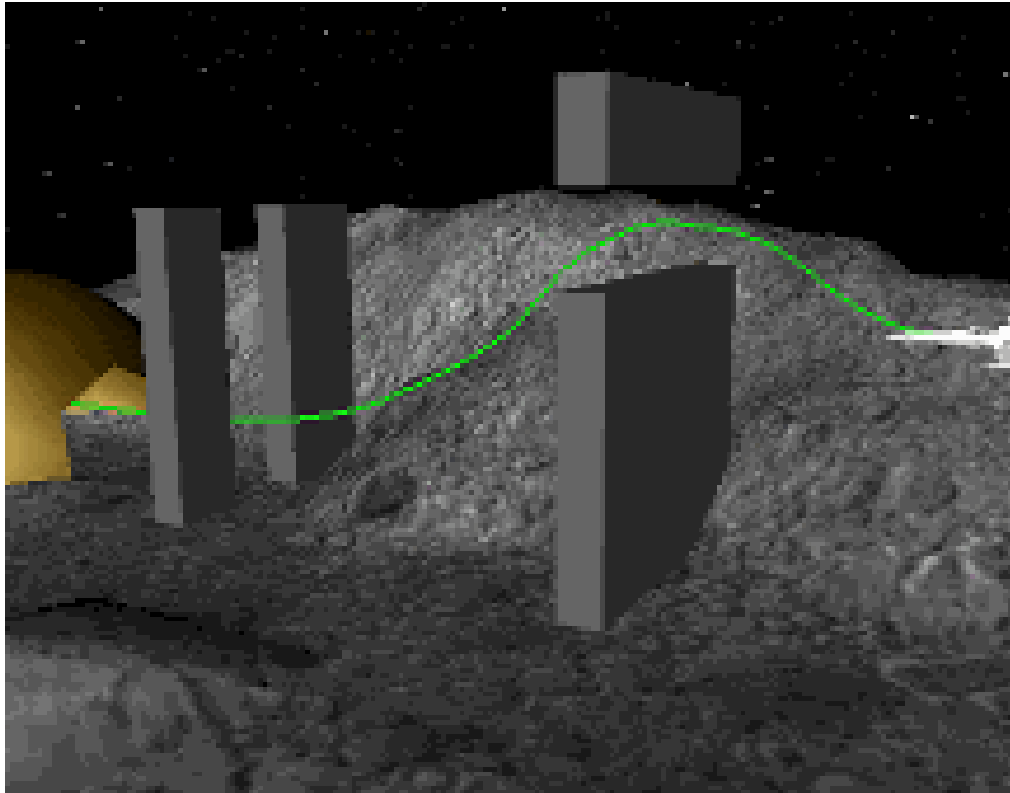
Kinodynamic Planning with RRT

- Kinematic RRT: Grow a tree of **feasible configuration paths** from the start to the goal configuration
 - Select a random / the goal configuration q , find the nearest configuration q_{near} to q , and greedily move from q_{near} to q .

Kinodynamic Planning with RRT

- Kinematic RRT: Grow a tree of **feasible configuration paths** from the start to the goal configuration
 - Select a random / the goal configuration q , find the nearest configuration q_{near} to q , and greedily move from q_{near} to q .
- Kinodynamic RRT: Grow a tree of feasible **control trajectories** $u(t), t \in \{0, \dots, T\}$ from the start to the goal state
 - Select a random / the goal configuration q , find the nearest configuration q_{near} to q , randomly select a control u , and execute u from q_{near} .

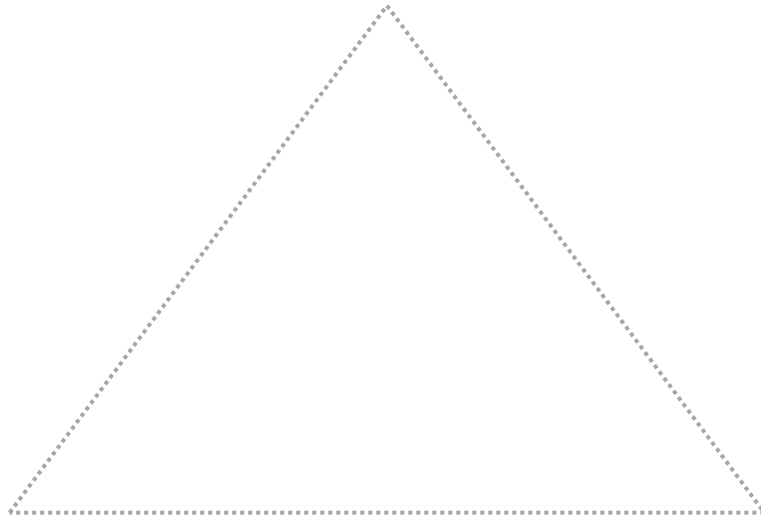
Kinodynamic Planning with RRT



Kinodynamic RRT Still Has Problems

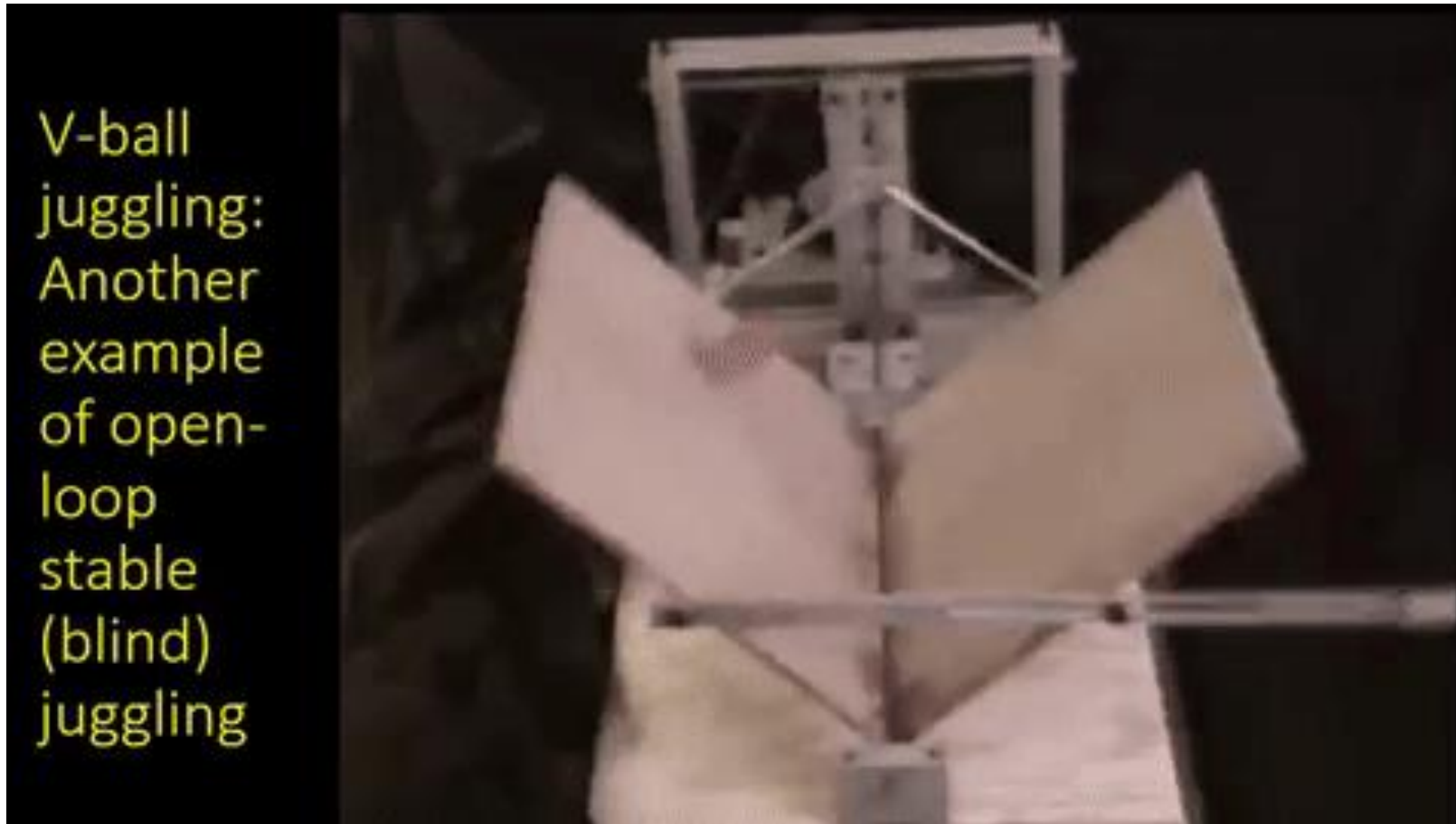
- Metric choice: The Euclidean distance between two states often correspond poorly with the length of exact control trajectories
 - We need to decide x_{near}
 - We need to decide if x_{new} is close enough to x
- Control choice: we need to sample a lot of u_{new} , which is sample inefficient
- Open-loop control: we decide the whole control trajectory from the initial state. We need to replan again the whole trajectory if something goes wrong...

Open-loop Control: Plan and Execute the Whole Trajectory without Feedback

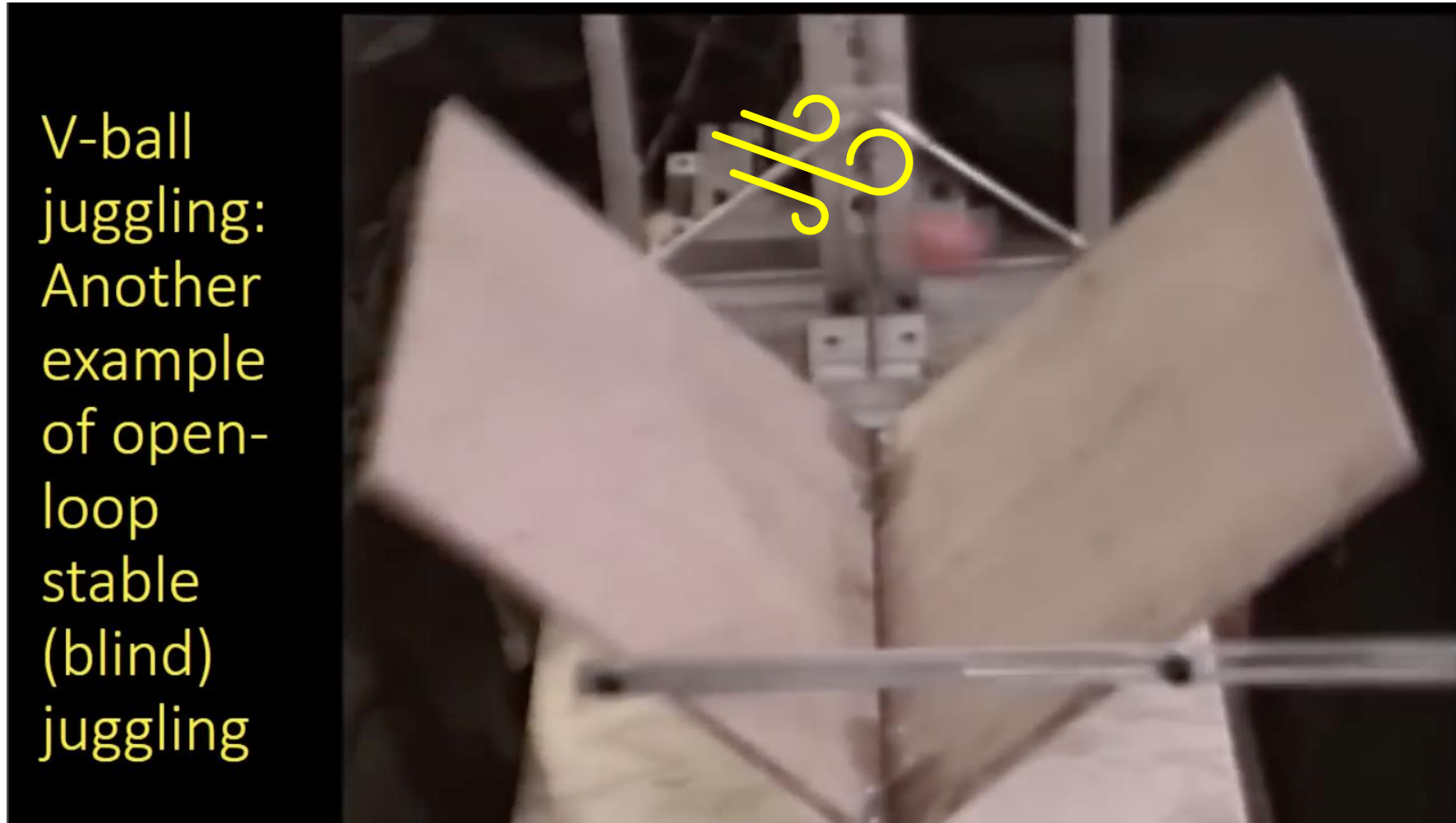


Open-loop Control: Plan and Execute the Whole Trajectory without Feedback

Open-loop Control for Robot Juggling



What if the Ball is Disturbed? We need to Re-decide the Control Inputs



Two Types of Control Strategy

Open-loop Control

Torque $\tau_1, \dots, \tau_T$



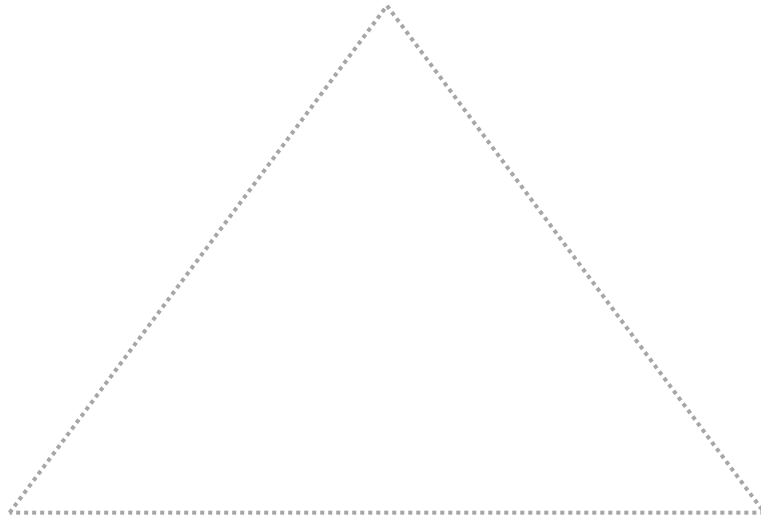
Close-loop Control

Torque τ_t



State s_{t+1}

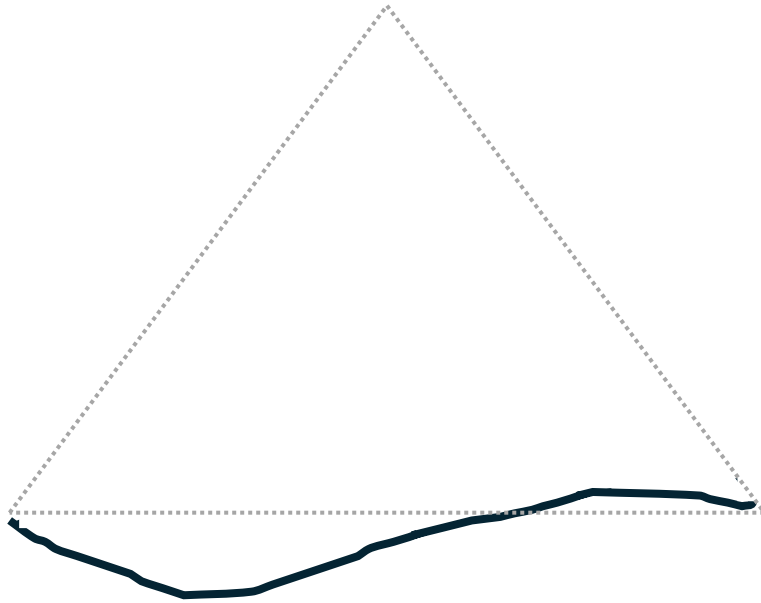
Close-loop Control: Repeat Planning and Executing the Trajectory with Feedback



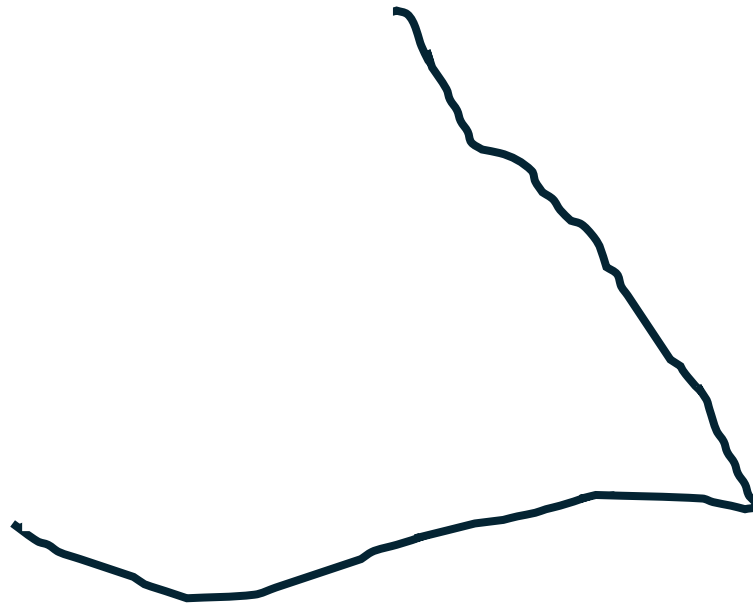
Close-loop Control: Repeat Planning and Executing the Trajectory with Feedback



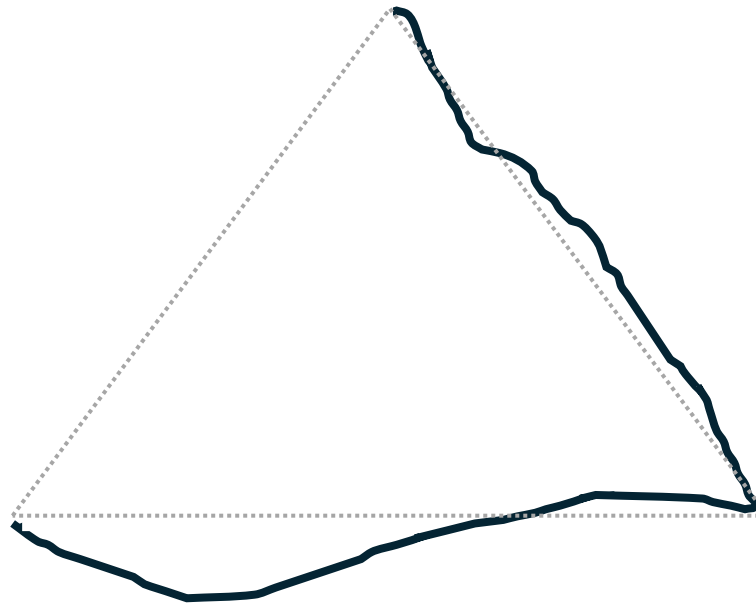
Close-loop Control: Repeat Planning and Executing the Trajectory with Feedback



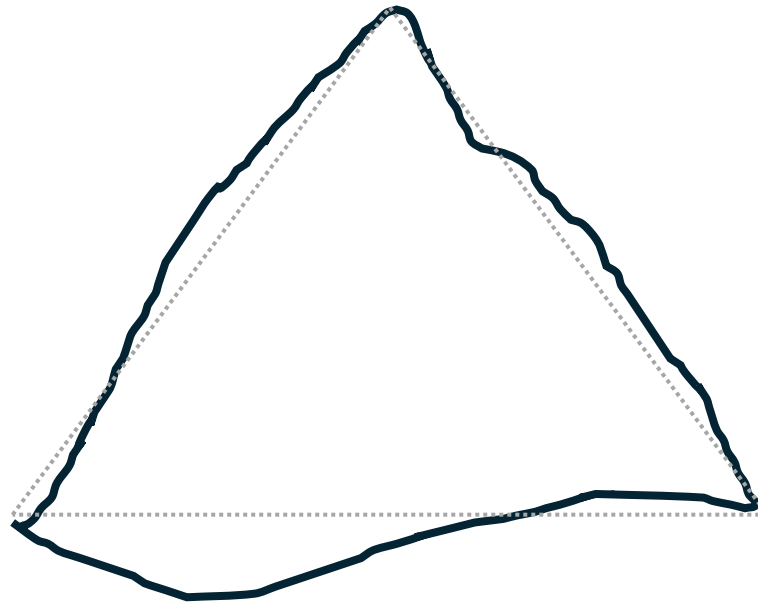
Close-loop Control: Repeat Planning and Executing the Trajectory with Feedback



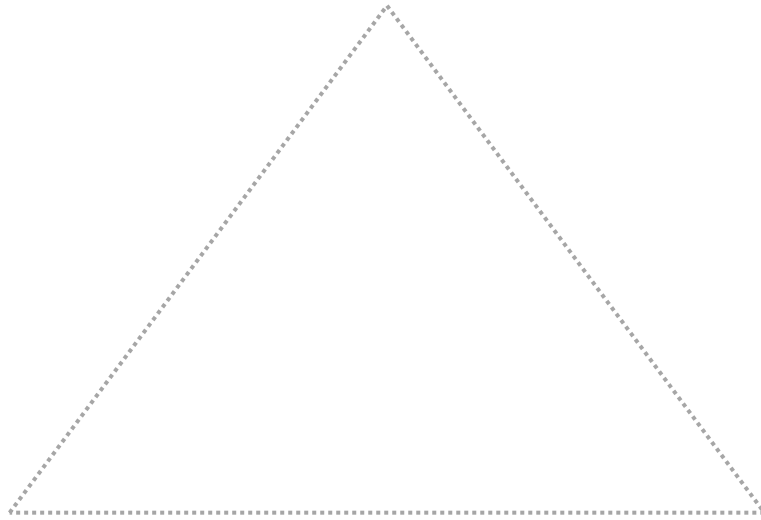
Close-loop Control: Repeat Planning and Executing the Trajectory with Feedback



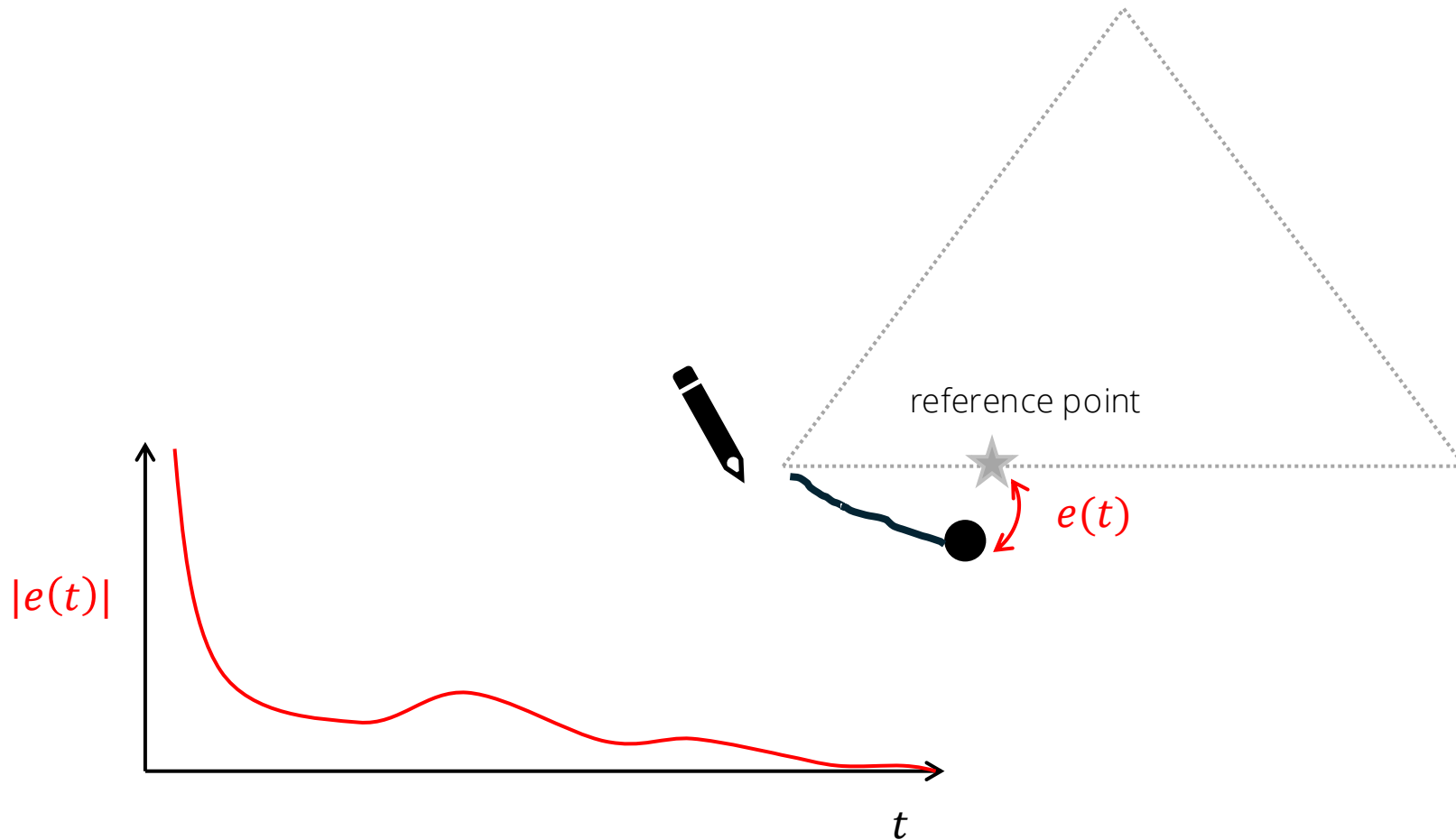
Close-loop Control: Repeat Planning and Executing the Trajectory with Feedback



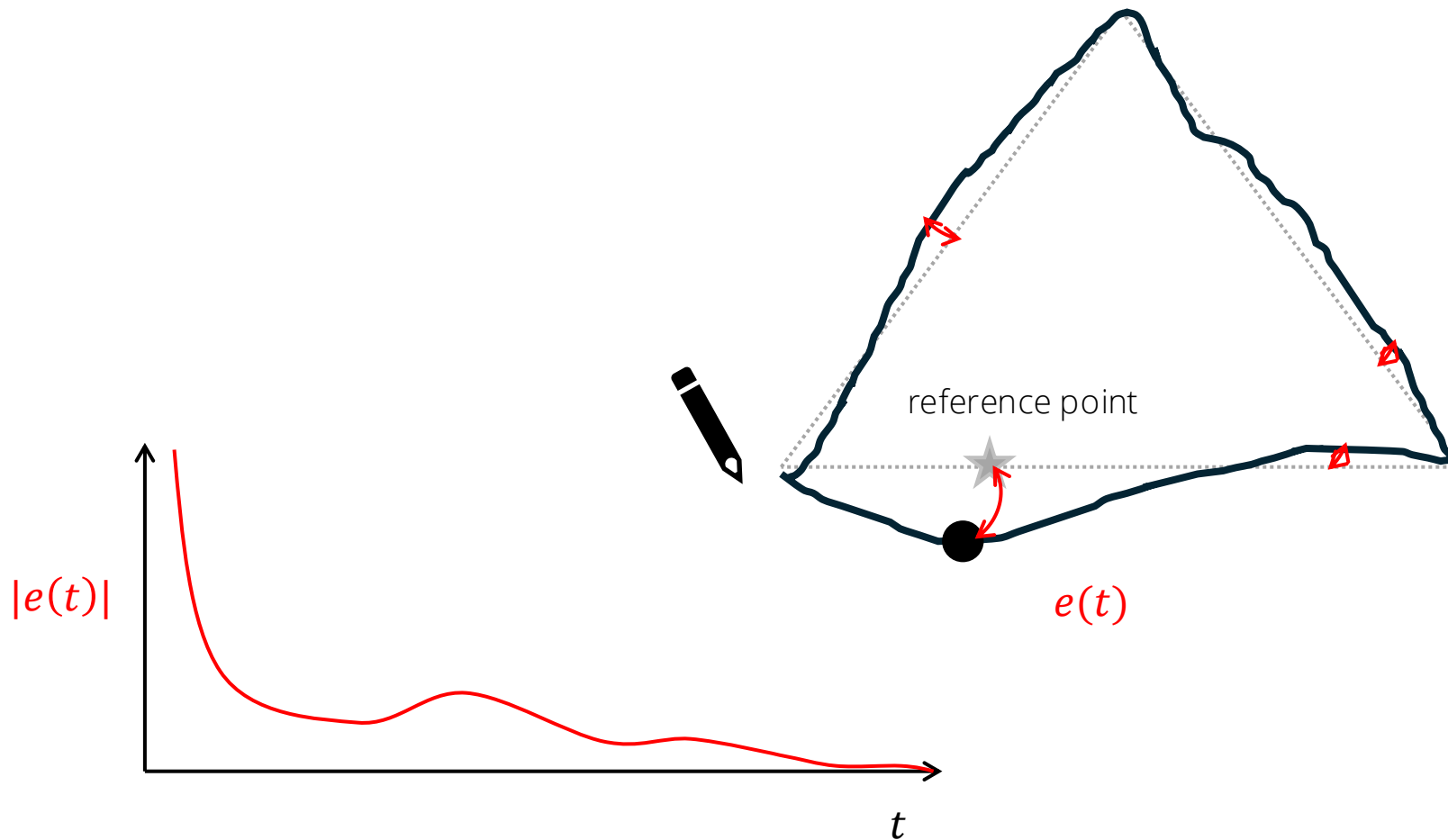
Feedback Control: Track a reference trajectory



Measure and Minimize Error to Reference Points

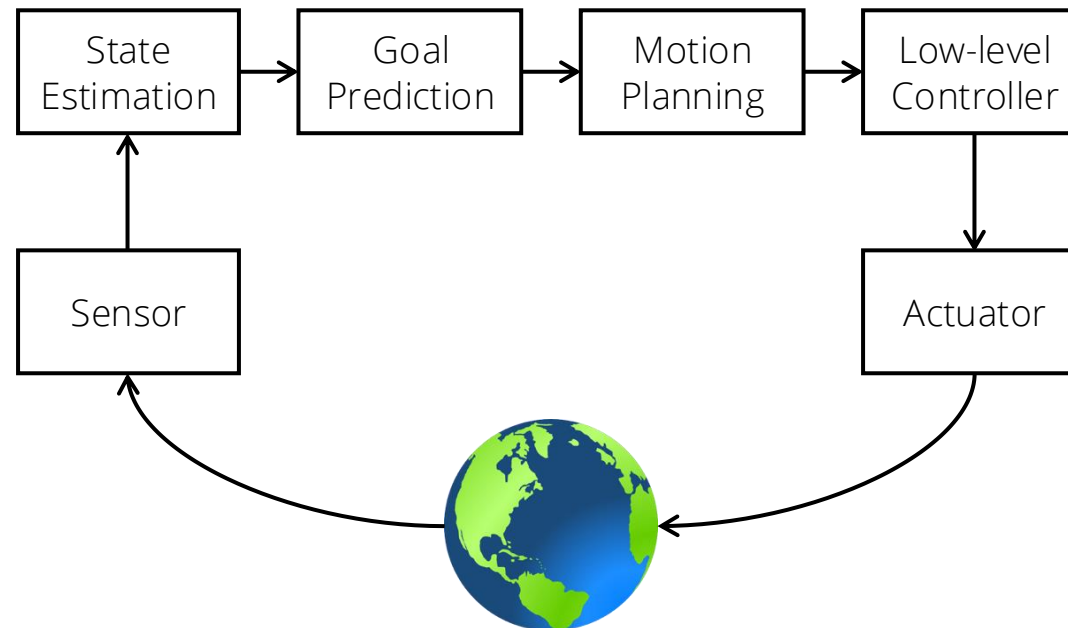


Measure and Minimize Error to Reference Points



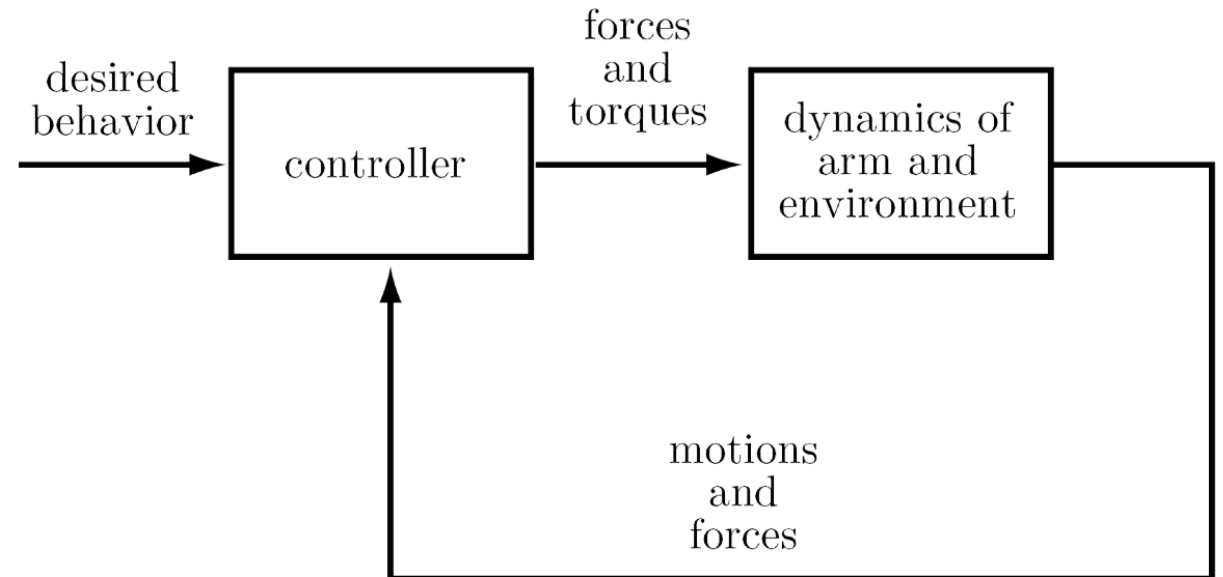
Basic Recipe of Planning and Control

1. Motion planning: compute a feasible path
 - grid search / PRM / RRT ...
2. Controller: predict a control to follow the path or minimize the "tracking" error.

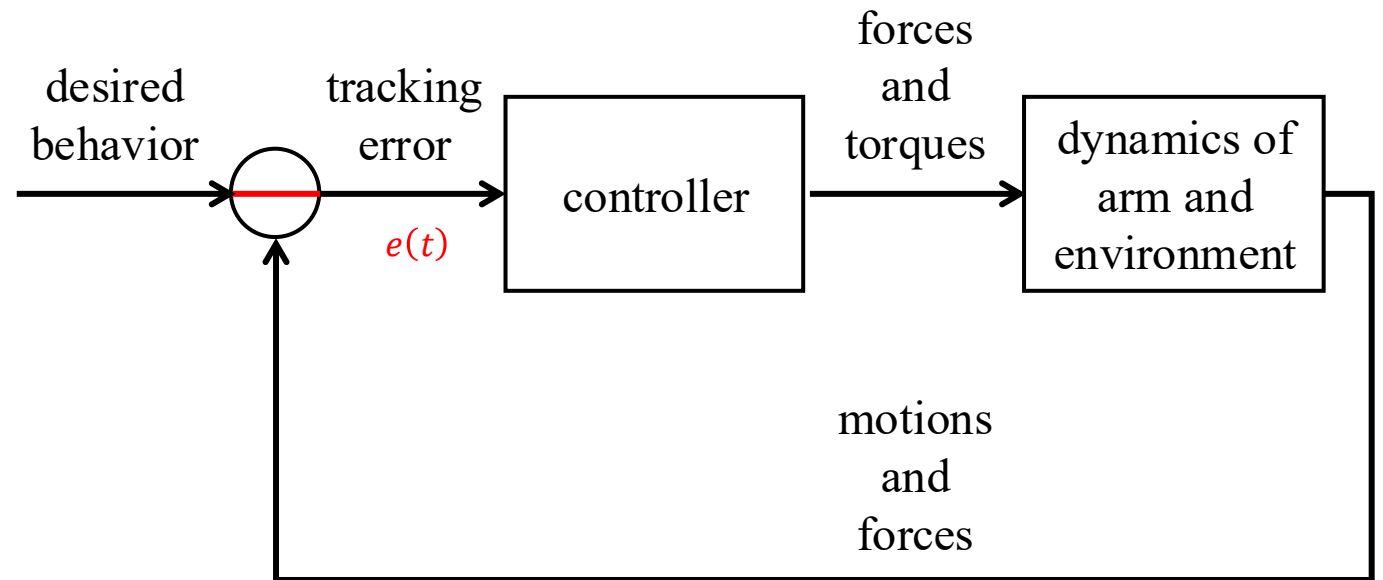
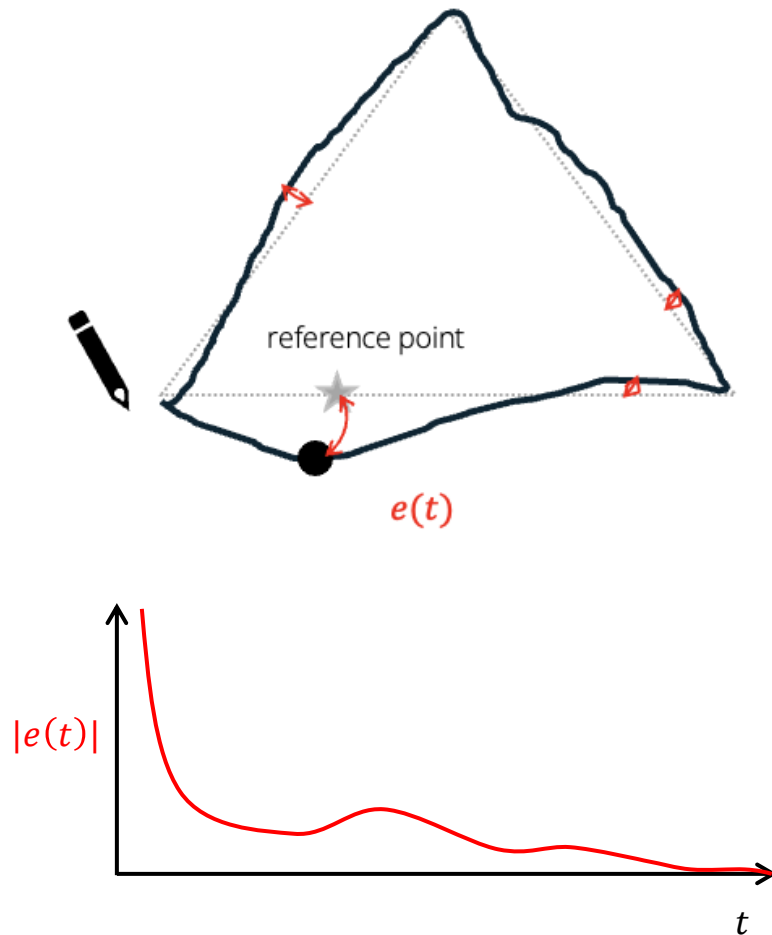


Control Systems

1. Controller: predict a control to follow the path / desired behavior
 - Feedback control
 - Optimal control
 - ...
2. Control $u(t)$ can be velocity / torque inputs



Feedback Controller: Continuously Adjust the Control Input to Minimize the Tracking Error

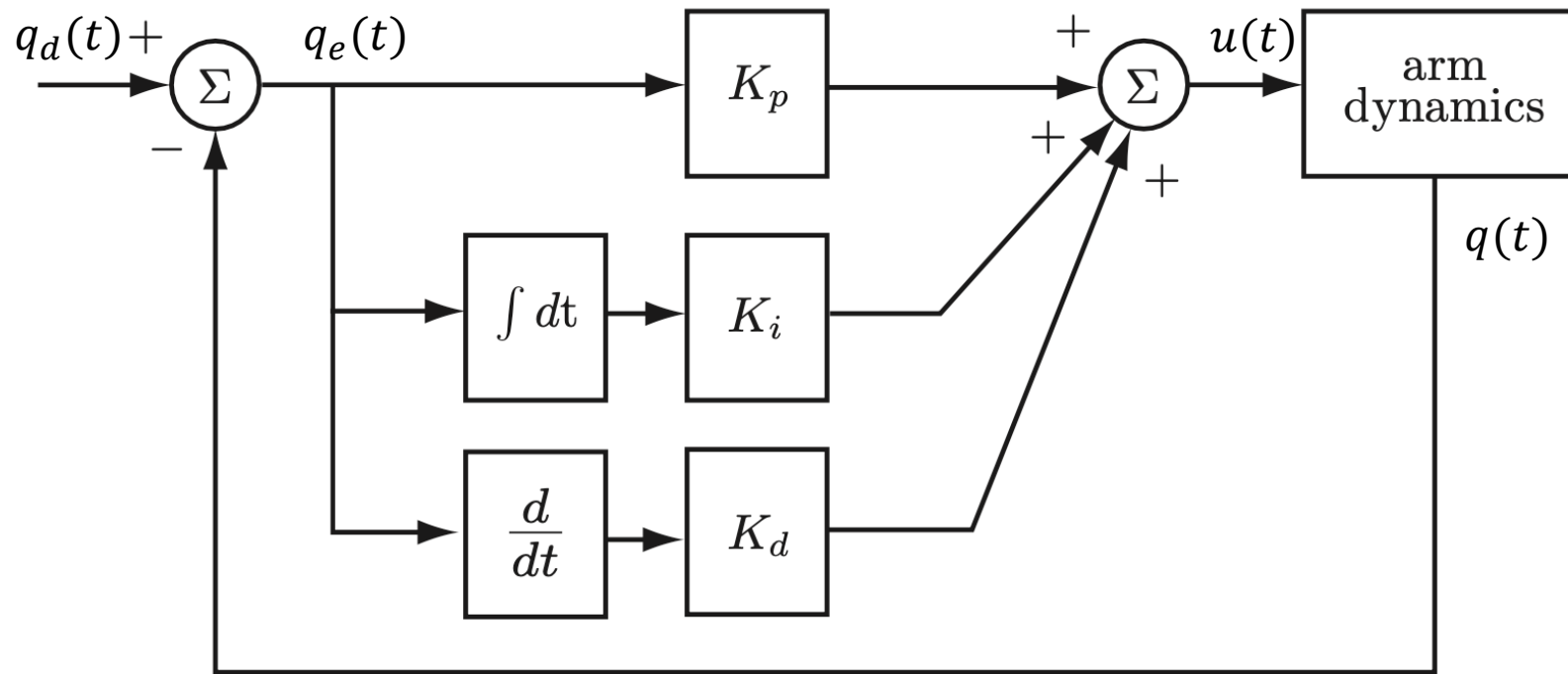


Content

- Open-loop control vs. close-loop control
- PID controllers
- Stability analysis of PID controllers
- Control in robotic manipulation
 - The effect of PD gain
 - Position control vs. force control
 - Joint-space control vs. task-space control
 - Compliant control (impedance control, admittance control)

A Simplified Block Diagram of PID Controller

- Control $u(t)$ can be velocity / torque inputs



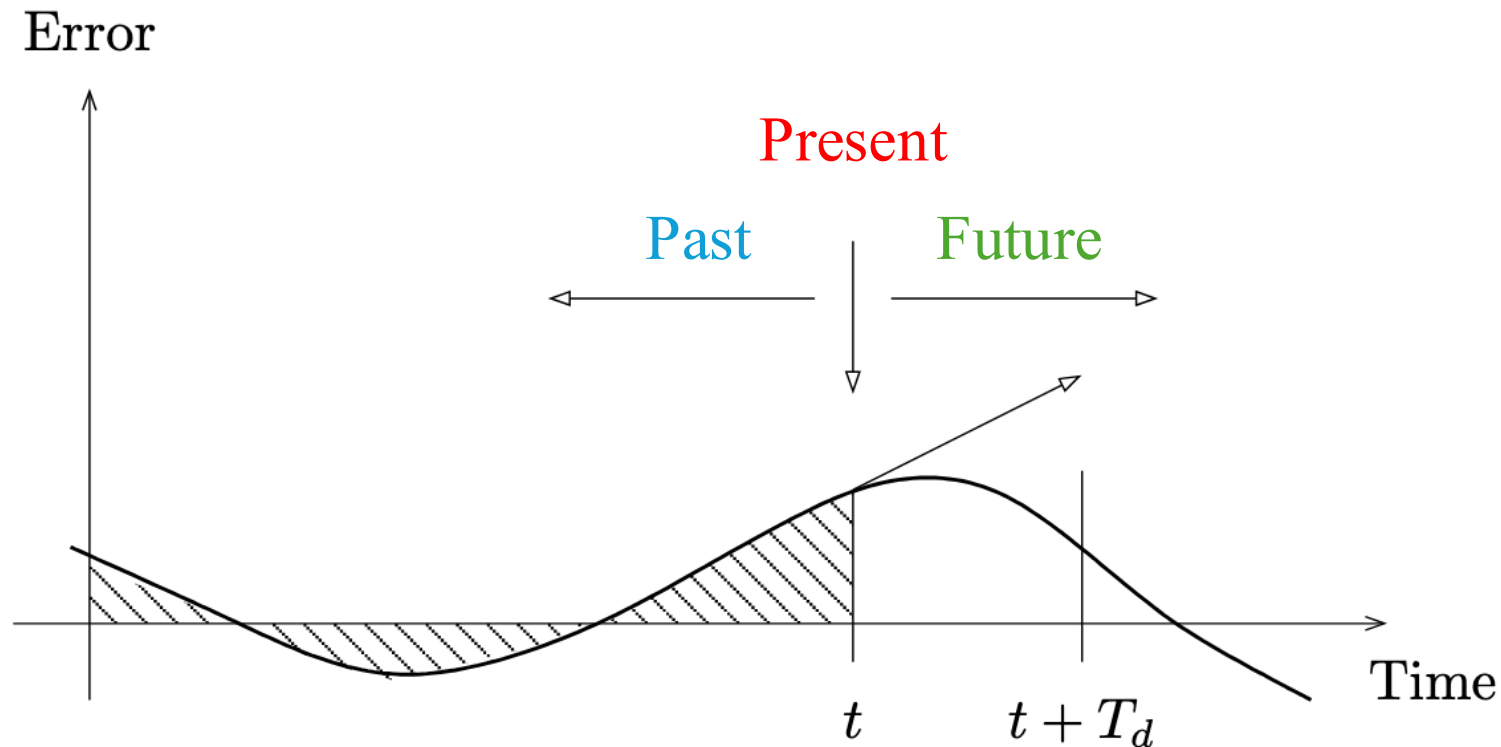
PID Feedback Controller

- PID feedback controller:
 - 1st-order error (immediate error)
 - Higher-order error (derivatives of errors)
 - Historical error (accumulated error)

$$u(t) = K_p q_e(t) + K_d \dot{q}_e(t) + K_i \int_0^t q_e(t) dt$$

- PID feedback controller:
 - 1st-order error (immediate error)
 - Higher-order error (derivatives of errors)
 - Historical error (accumulated error)

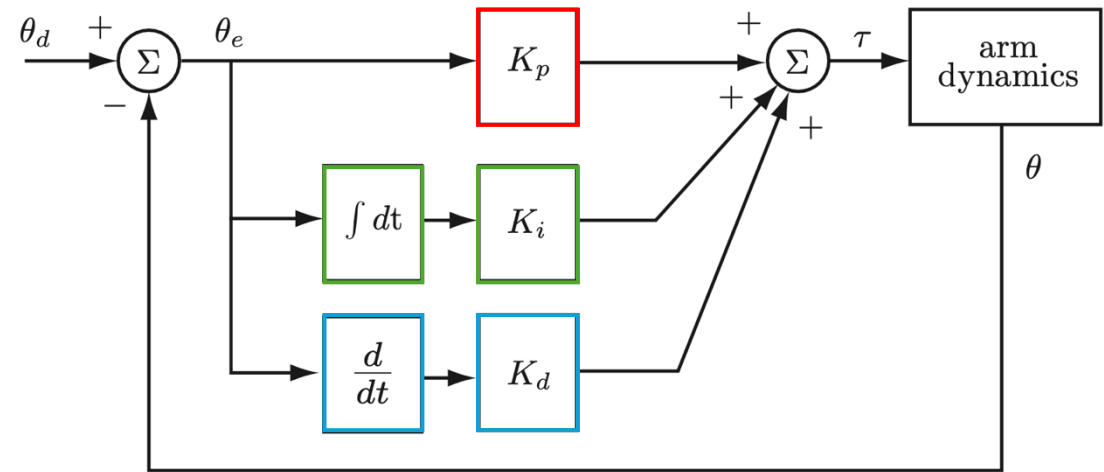
$$u(t) = K_p q_e(t) + K_d \dot{q}_e(t) + K_i \int_0^t q_e(t) dt$$



An Overview of PID Controller



Image generated by Gemini



- Current temperature difference (immediate error)
- Future temperature difference (derivatives of errors)
- Accumulated temperature difference (accumulated error)

Demo Time

Content

- Open-loop control vs. close-loop control
- PID controllers
- Stability analysis of PID controllers
- Control in robotic manipulation
 - The effect of PD gain
 - Joint-space control vs. task-space control
 - Position control vs. force control
 - Indirect force control (impedance control)

We Need to Analyze the Stability of PID Controller

- Simple cases: control input is always zero
- Complex cases: control input is non-zero

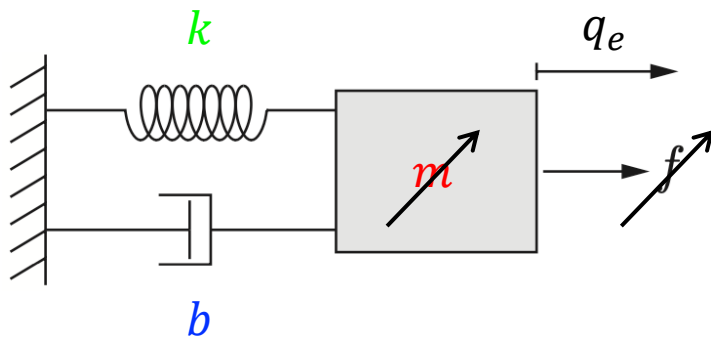
Linear Error Dynamics

- Let's define $q_e(t) = q_d(t) - q(t)$ as the tracking error, where $q_d(t)$ is the desired behavior and $q(t)$ is the current behavior
- The purpose of the feedback controller is to create an error dynamics such that $q_e(t)$ tends to a small value, as t increases

1st-order Error Dynamics

- Take linear mass-spring-damper for example, let $m = 0$ and $f = 0$:

$$b\dot{q}_e(t) + kq_e(t) = 0 \Rightarrow \dot{q}_e(t) + \frac{k}{b}q_e(t) = 0$$

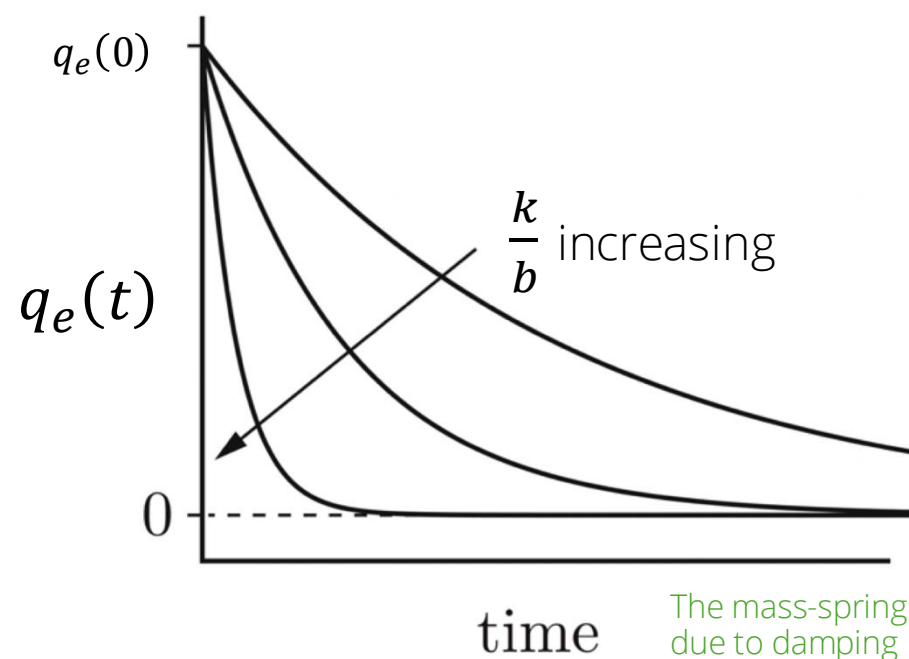
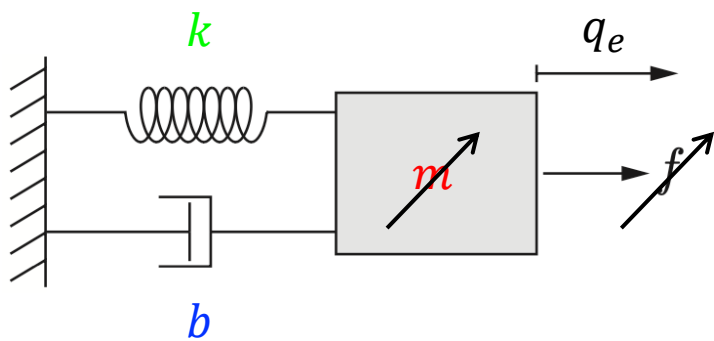


1st-order Error Dynamics

- Take linear mass-spring-damper for example, let $m = 0$ and $f = 0$:

$$b\dot{q}_e(t) + kq_e(t) = 0 \Rightarrow \dot{q}_e(t) + \frac{k}{b}q_e(t) = 0$$

- We have solution $q_e(t) = e^{-\frac{k}{b}t}q_e(0)$

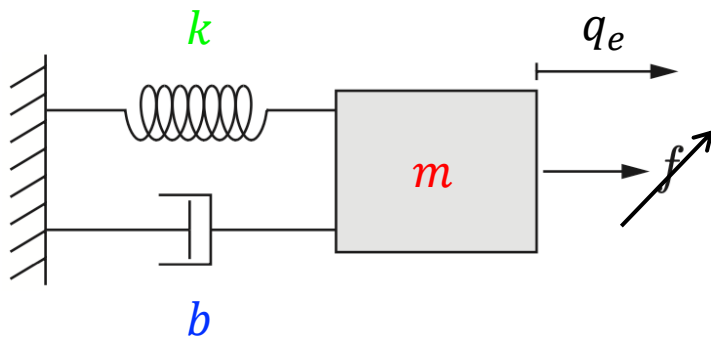


The mass-spring stops due to damping

2nd-order Error Dynamics

- Take linear mass-spring-damper for example, let $f = 0$:

$$m\ddot{q}_e(t) + b\dot{q}_e(t) + kq_e(t) = 0 \Rightarrow \ddot{q}_e(t) + \frac{b}{m}\dot{q}_e(t) + \frac{k}{m}q_e(t) = 0$$
$$\Rightarrow \ddot{q}_e(t) + 2\xi\omega_n\dot{q}_e(t) + \omega_n^2q_e(t) = 0$$



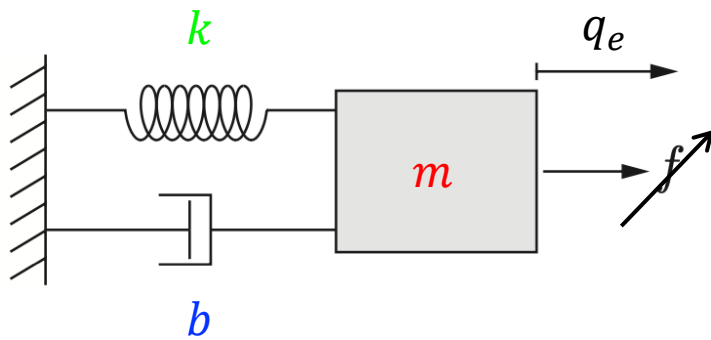
$$\omega_n = \sqrt{\frac{k}{m}} \text{ is known as the natural frequency}$$
$$\xi = \frac{b}{2\sqrt{km}} \text{ is known as the damping ratio}$$

2nd-order Error Dynamics

- Take linear mass-spring-damper for example, let $f = 0$:

$$m\ddot{q}_e(t) + b\dot{q}_e(t) + kq_e(t) = 0 \Rightarrow \ddot{q}_e(t) + \frac{b}{m}\dot{q}_e(t) + \frac{k}{m}q_e(t) = 0$$

$$\Rightarrow \ddot{q}_e(t) + 2\xi\omega_n\dot{q}_e(t) + \omega_n^2q_e(t) = 0$$



The characteristic polynomial:

$$s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$$

Two roots:

$$s_1 = -\zeta\omega_n + \omega_n\sqrt{\zeta^2 - 1}$$

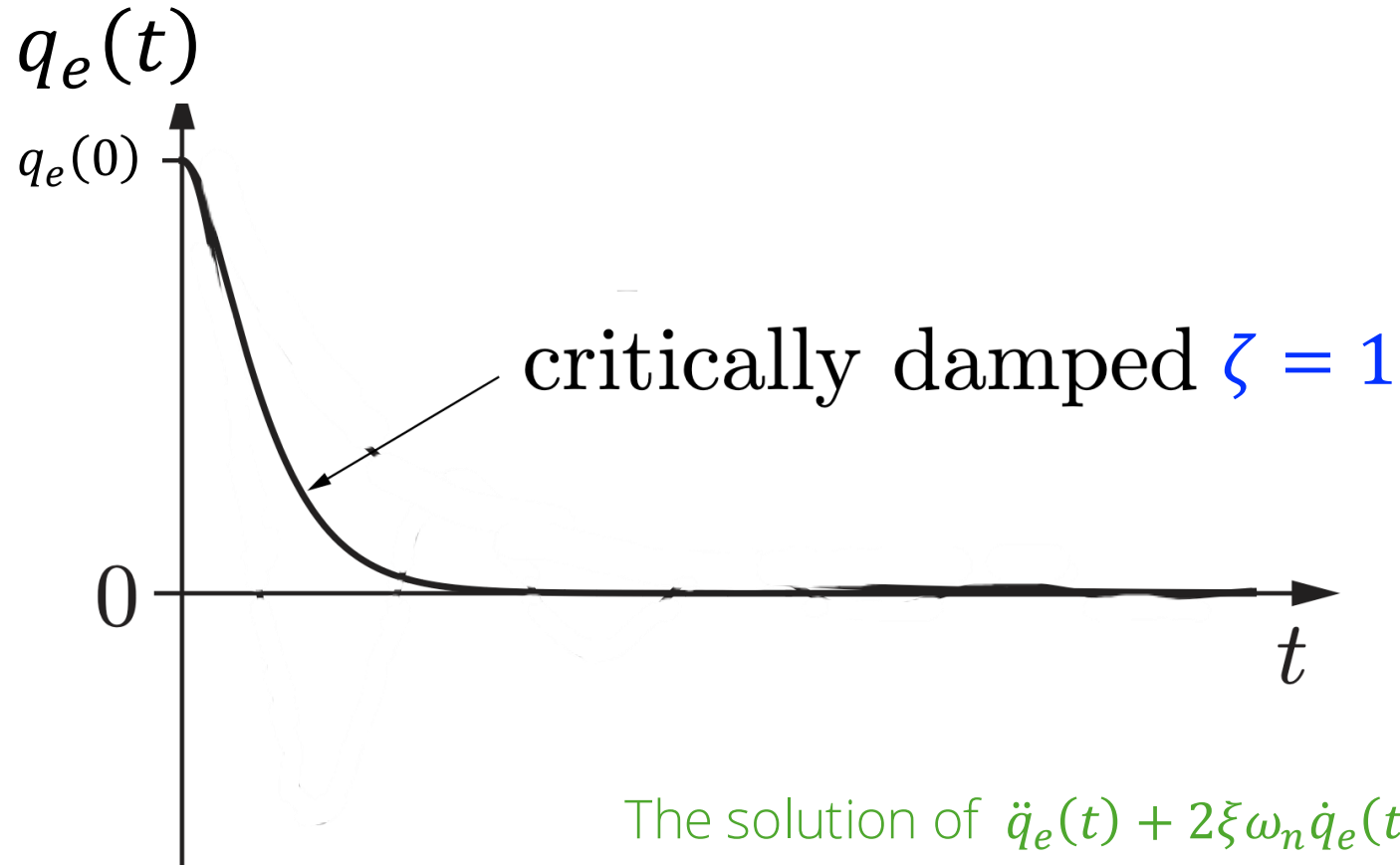
$$s_2 = -\zeta\omega_n - \omega_n\sqrt{\zeta^2 - 1}.$$

Stable conditions:

$$\zeta\omega_n > 0$$

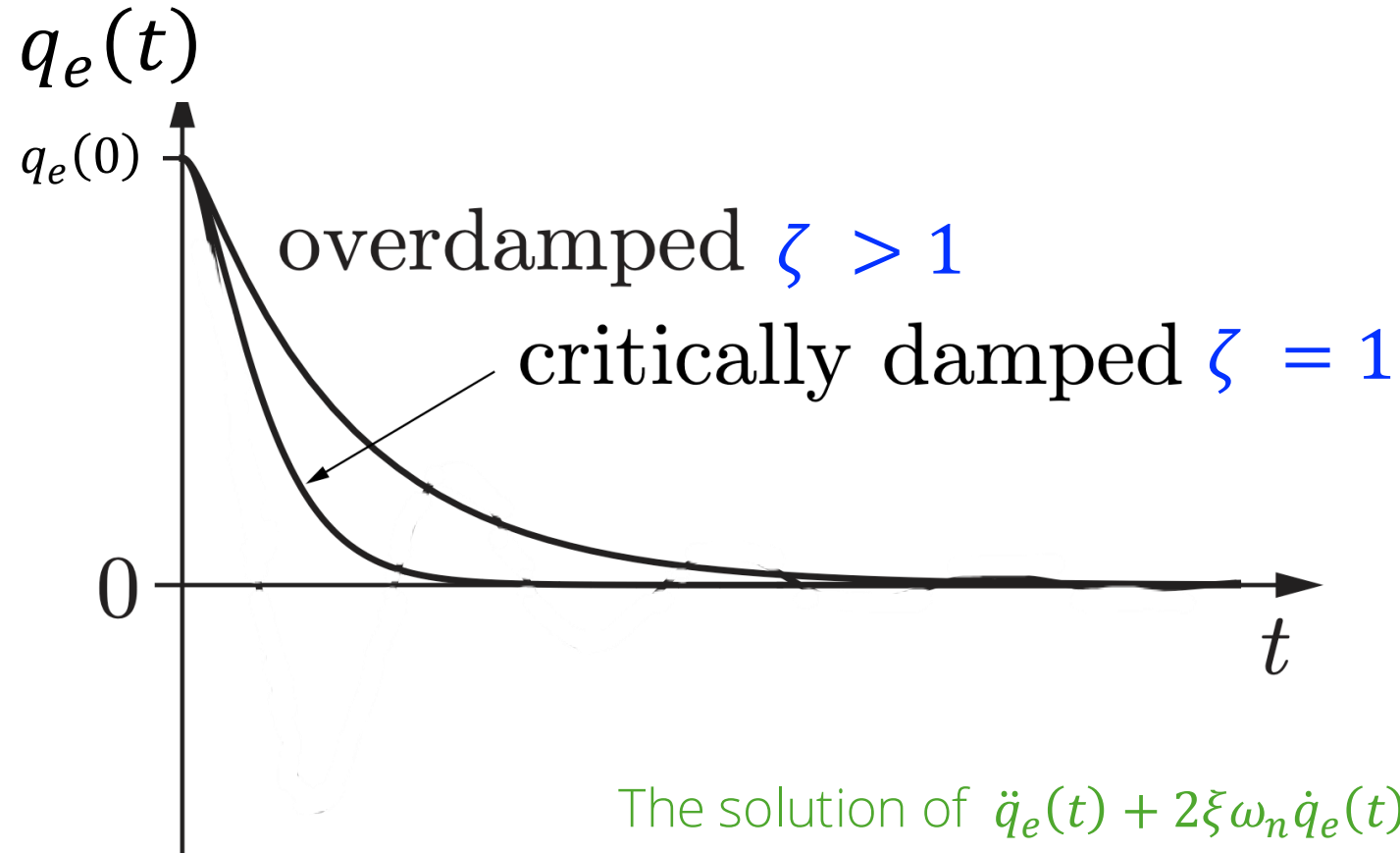
$$\omega_n^2 > 0.$$

2nd-order Error Dynamics



The solution of $\ddot{q}_e(t) + 2\xi\omega_n\dot{q}_e(t) + \omega_n^2q_e(t) = 0$:
 $q_e(t) = (c_1 + c_2t)e^{-\omega_nt}$

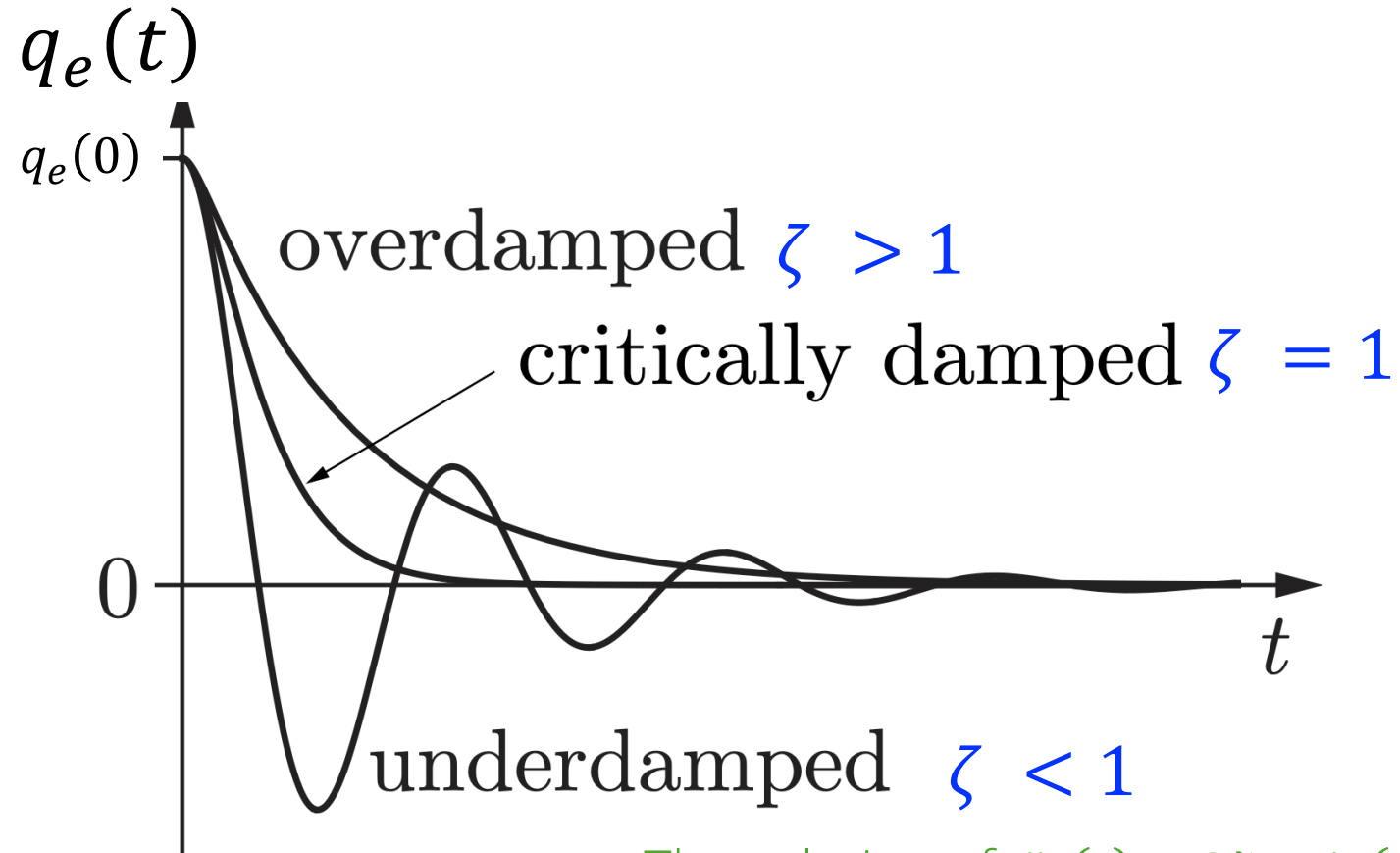
2nd-order Error Dynamics



The solution of $\ddot{q}_e(t) + 2\zeta\omega_n\dot{q}_e(t) + \omega_n^2q_e(t) = 0$:

$$q_e(t) = c_1e^{s_1t} + c_2e^{s_2t}$$

2nd-order Error Dynamics



The solution of $\ddot{q}_e(t) + 2\zeta\omega_n\dot{q}_e(t) + \omega_n^2 q_e(t) = 0$:

$$q_e(t) = \left(c_1 \cos \omega_n \sqrt{1 - \zeta^2} t + c_2 \sin \omega_n \sqrt{1 - \zeta^2} t \right) e^{-\zeta \omega_n t}$$

We Need to Analyze the Stability of PID Controller

- Simple cases: control input is always zero
- Complex cases: control input is non-zero

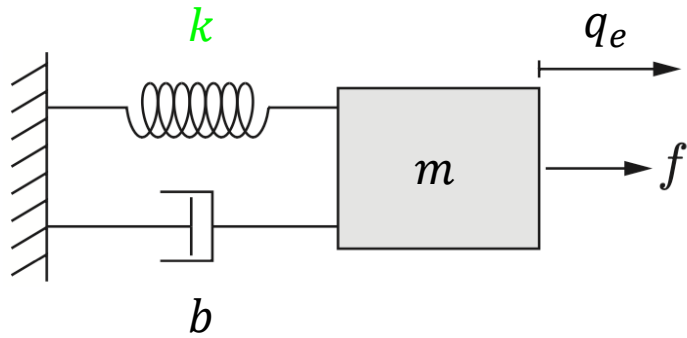
Proportional Controller: Minimize Immediate Error

- Control with velocity inputs:

$$u(t) = \dot{q}(t) = K_p (q_d(t) - q(t)) = K_p q_e(t)$$

↓
desired $q(t)$

- The constant control gain K_p acts somewhat like a virtual spring



Proportional Controller: Minimize Immediate Error

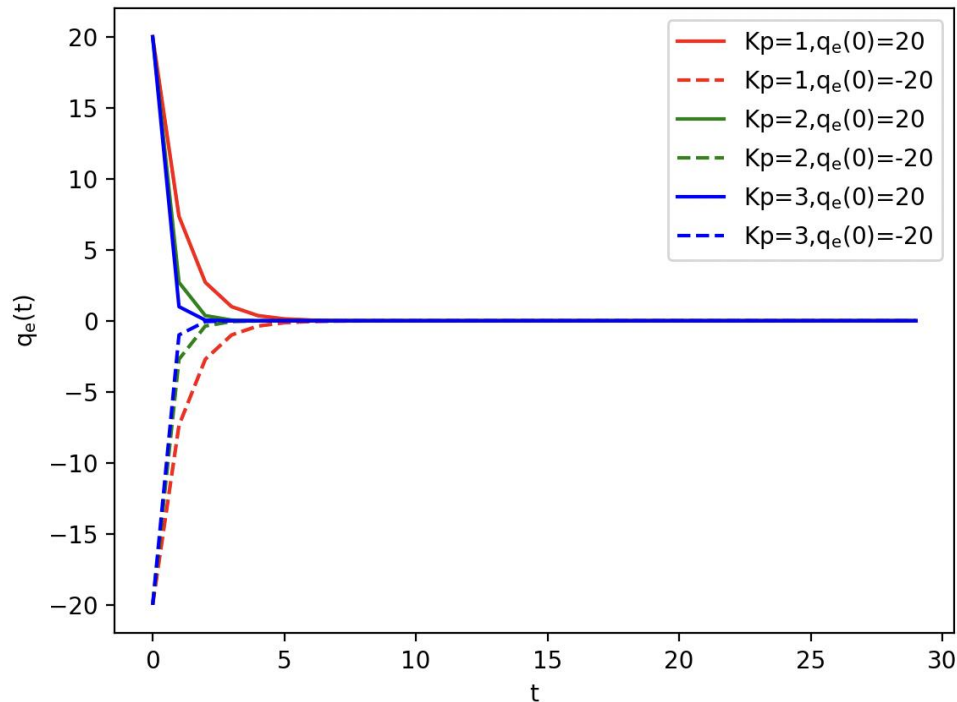
- Control with velocity inputs:

$$u(t) = \dot{q}(t) = K_p(q_d(t) - q(t)) = K_p q_e(t)$$

- Case 1: if $q_d(t)$ is constant

$$\dot{q}_e(t) = \dot{q}_d(t) - \dot{q}(t) \Rightarrow \dot{q}(t) = -\dot{q}_e(t)$$

$$-\dot{q}_e(t) = K_p q_e(t) \Rightarrow q_e(t) = e^{-K_p t} q_e(0)$$



Proportional Controller: Minimize Immediate Error

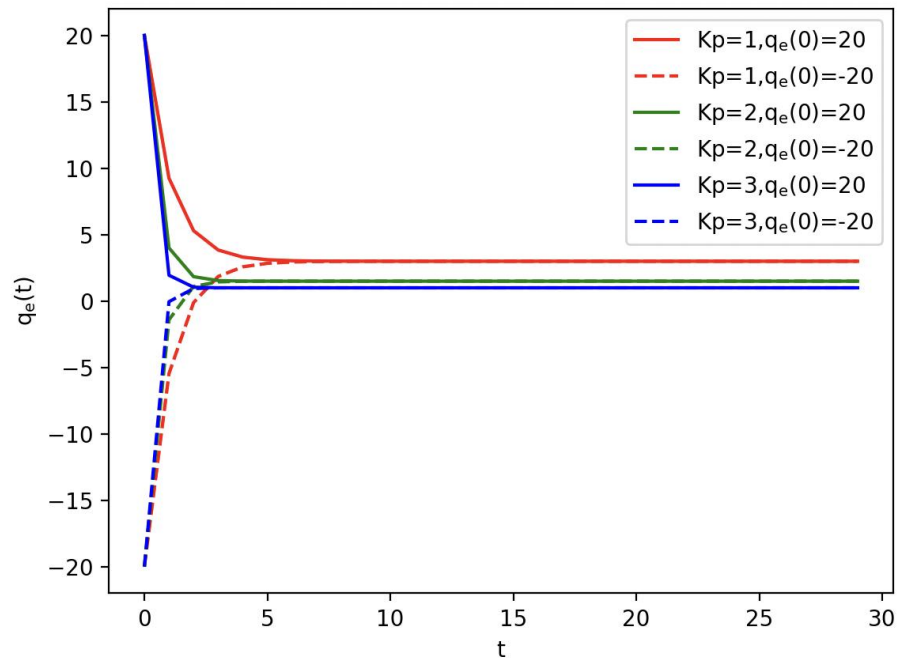
- Control with velocity inputs:

$$u(t) = \dot{q}(t) = K_p(q_d(t) - q(t)) = K_p q_e(t)$$

- Case 2: if $q_d(t)$ is not constant, but $\dot{q}_d(t)$ is constant

$$\dot{q}_d(t) = c$$

$$\dot{q}_e(t) = c - \dot{q}(t) = c - K_p q_e(t) \Rightarrow q_e(t) = \frac{c}{K_p} + (q_e(0) - \frac{c}{K_p})e^{-K_p t}$$



Proportional Controller: Minimize Immediate Error

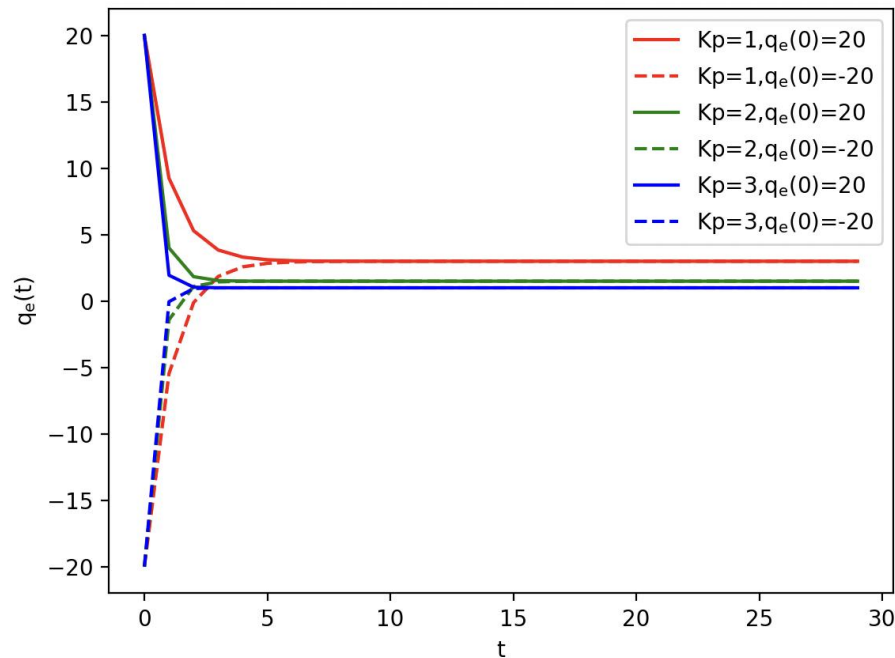
- Control with velocity inputs:

$$u(t) = \dot{q}(t) = K_p(q_d(t) - q(t)) = K_p q_e(t)$$

- Case 2: if $q_d(t)$ is not constant, but $\dot{q}_d(t)$ is constant

$$\dot{q}_d(t) = c$$

$$\dot{q}_e(t) = c - \dot{q}(t) = c - K_p q_e(t) \Rightarrow q_e(t) = \frac{c}{K_p} + (q_e(0) - \frac{c}{K_p})e^{-K_p t}$$



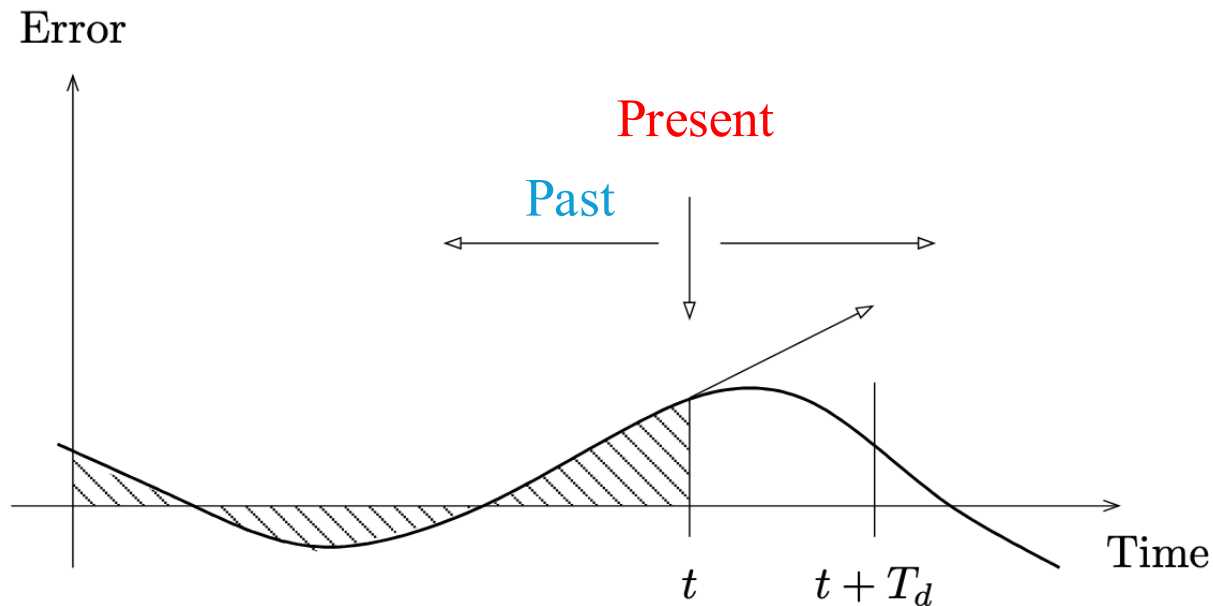
Solution?

- Increase K_p . What if K_p is bounded?
- Minimize accumulated error

Proportional Integral Controller: Minimize Historical Error

- Control with velocity inputs:

$$\dot{q}(t) = K_p q_e(t) + K_i \int_0^t q_e(t) dt$$



Proportional Integral Controller: Minimize Historical Error

- Control with velocity inputs:

$$u(t) = \dot{q}(t) = K_p q_e(t) + K_i \int_0^t q_e(t) dt$$

- If $\dot{q}_d(t)$ is constant

$$\dot{q}(t) = \dot{q}_d(t) - \dot{q}_e(t) = K_p q_e(t) + K_i \int_0^t q_e(t) dt$$

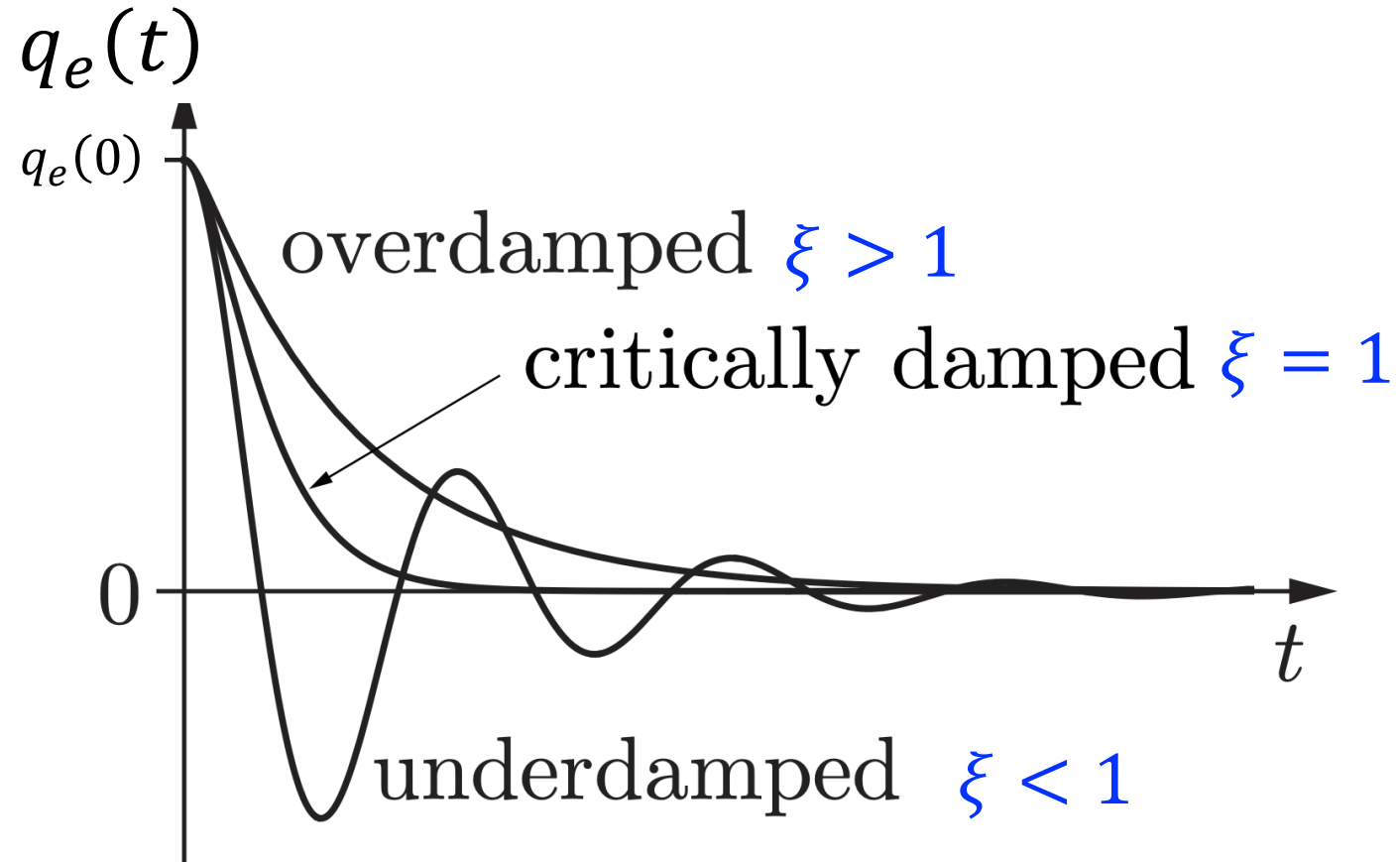
$$\Rightarrow c = \dot{q}_e(t) + K_p q_e(t) + K_i \int_0^t q_e(t) dt$$

(Take derivatives)

$$\Rightarrow \ddot{q}_e(t) + K_p \dot{q}_e(t) + K_i q_e(t) = 0$$

second-order error dynamics

2nd-order Error Dynamics



2nd-order Error Dynamics

- Given the standard second-order form:

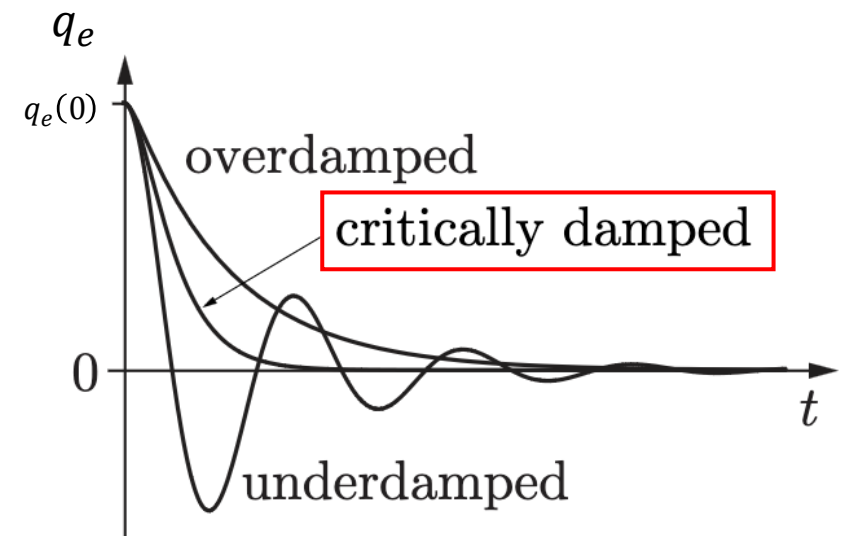
$$\ddot{q}_e(t) + 2\zeta\omega_n\dot{q}_e(t) + \omega_n^2 q_e(t) = 0$$

- Our PI controller:

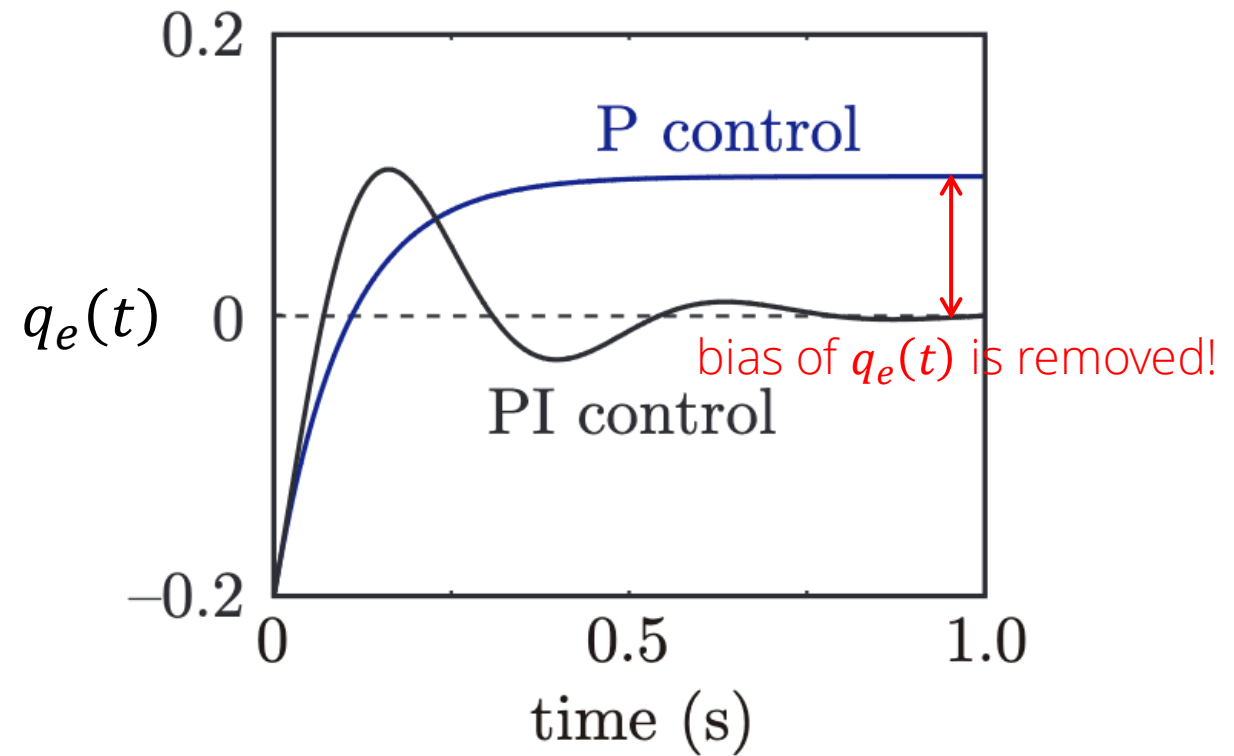
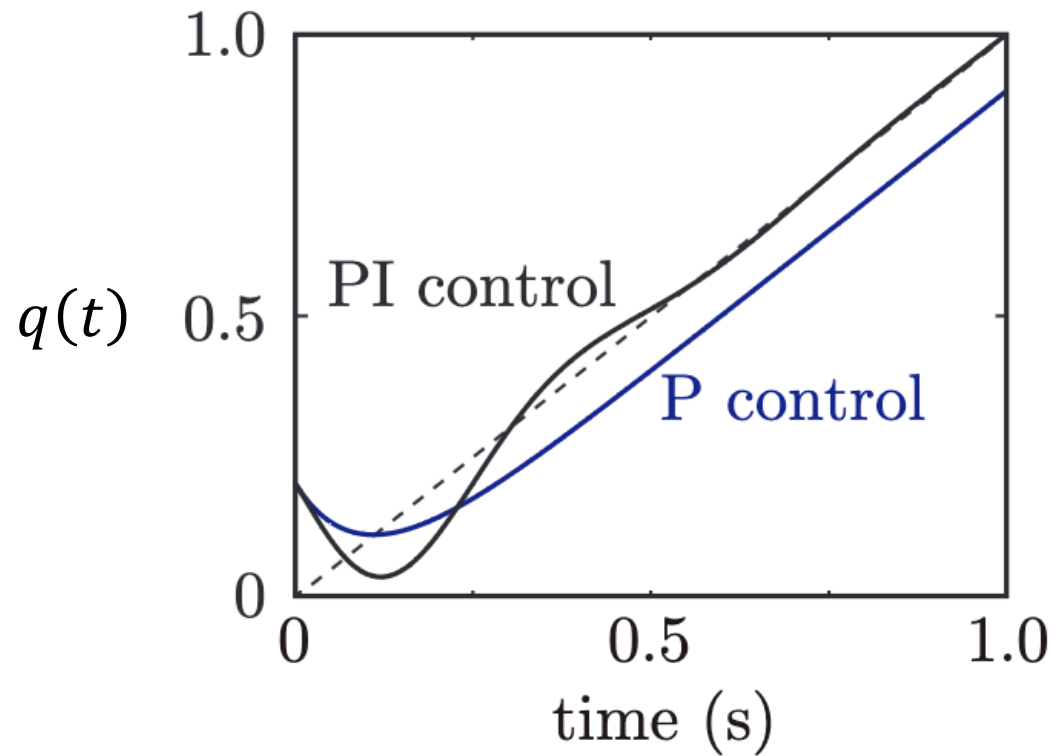
$$\ddot{q}_e(t) + K_p\dot{q}_e(t) + K_i q_e(t) = 0$$

- We have $\omega_n = \sqrt{K_i}$ and $\zeta = \frac{K_p}{2\sqrt{K_i}}$
- To achieve critically damped state, we need $\zeta = 1$, $\zeta\omega_n > 0$ and $\omega_n^2 > 0$

Need to tune carefully!



PI Controller vs. P Controller

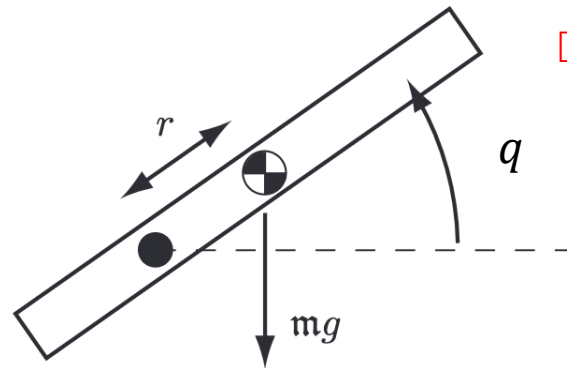


How About Force Inputs?

- Velocity inputs are limited to applications with low/predictable force-torque requirements. Here, we consider control force inputs.
- Let's take a single-joint robot for example

$$\tau = M\ddot{q}(t) + mgr \cos q(t) + \underbrace{b\dot{q}(t)}$$

Damping (e.g friction)



- PID controller: $\tau = K_p q_e(t) + K_d \dot{q}_e(t) + K_i \int_0^t q_e(t) dt$

[Check the Appendix for the more detailed derivation](#)

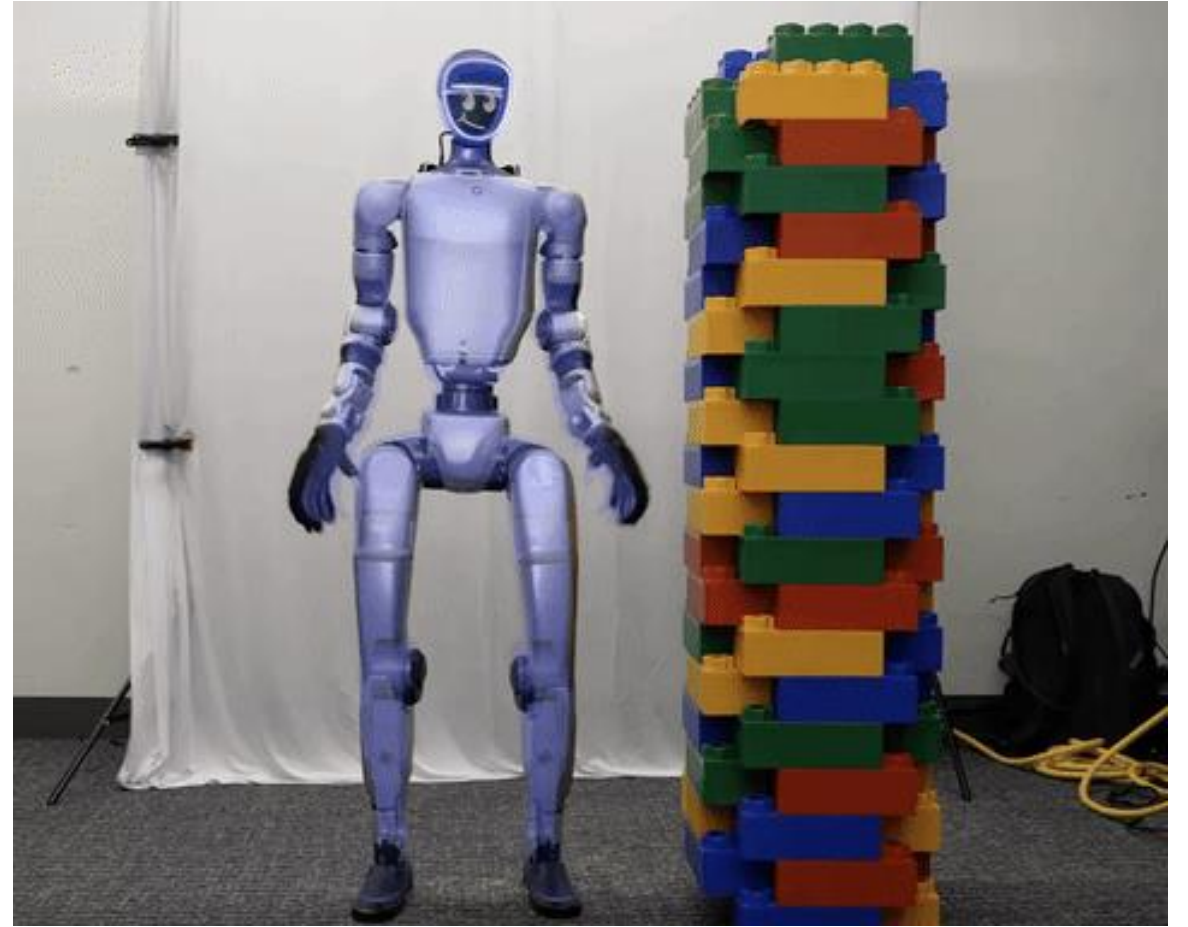
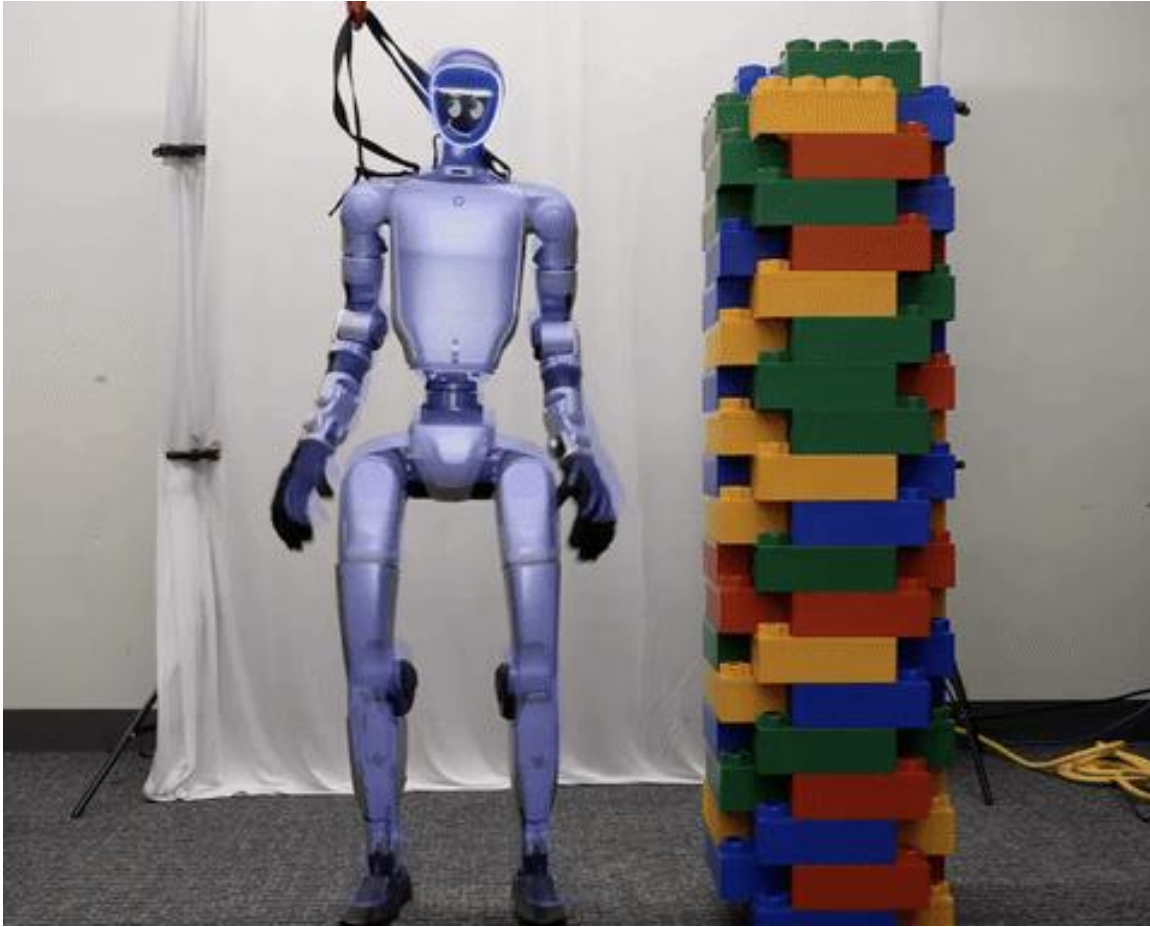
Content

- Open-loop control vs. close-loop control
- PID controllers
- Stability analysis of PID controllers
- Control in robotic manipulation
 - The effect of PD gain
 - Joint-space control vs. task-space control
 - Position control vs. force control
 - Indirect force control (impedance control)

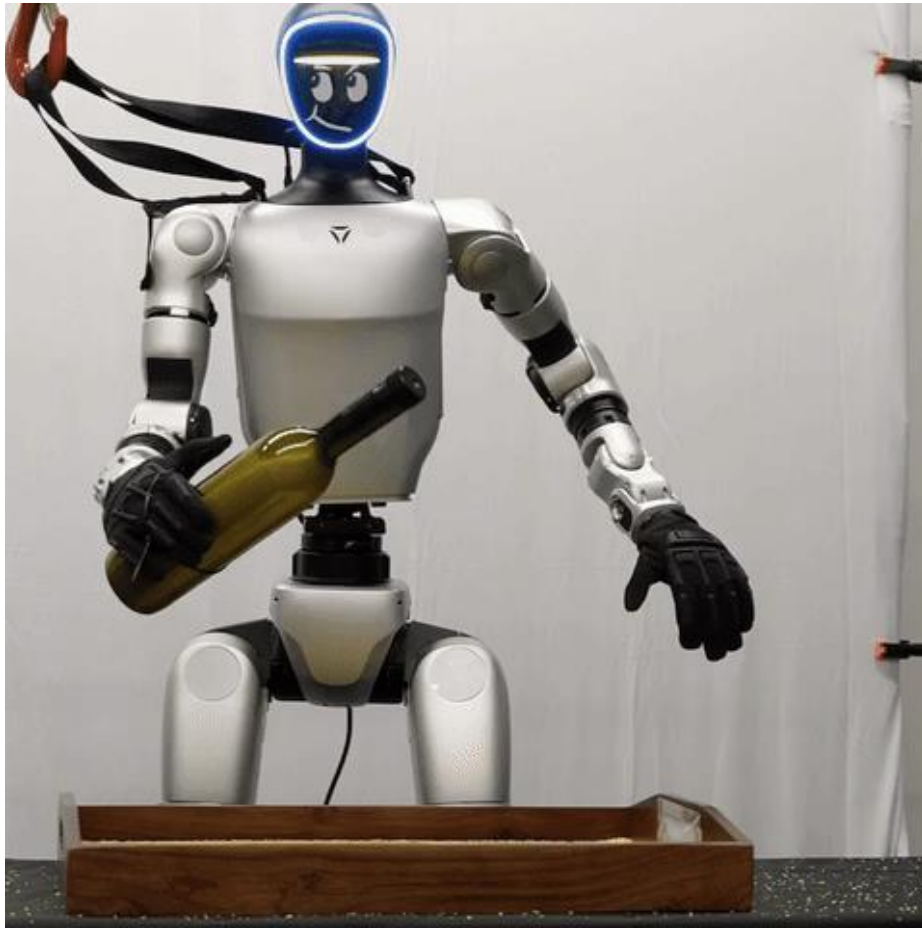
What Makes the Robot Behaves So Differently?



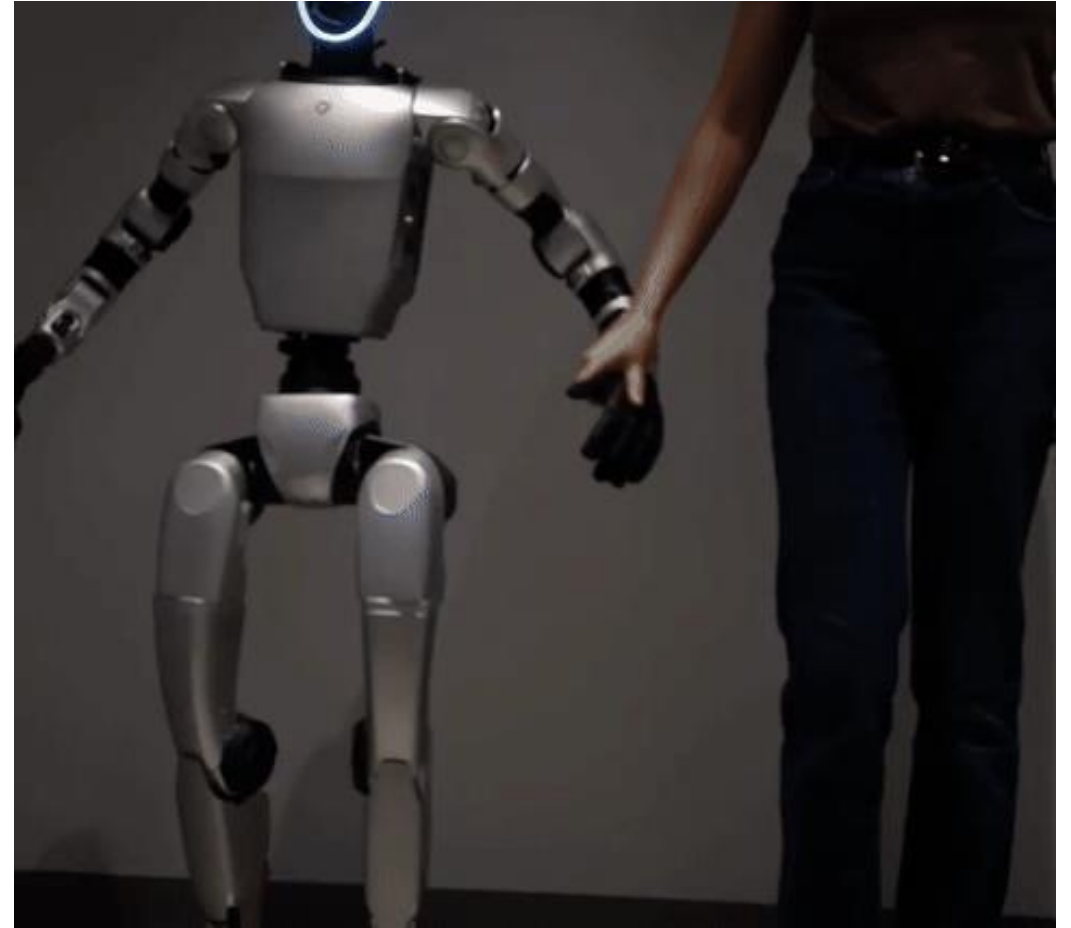
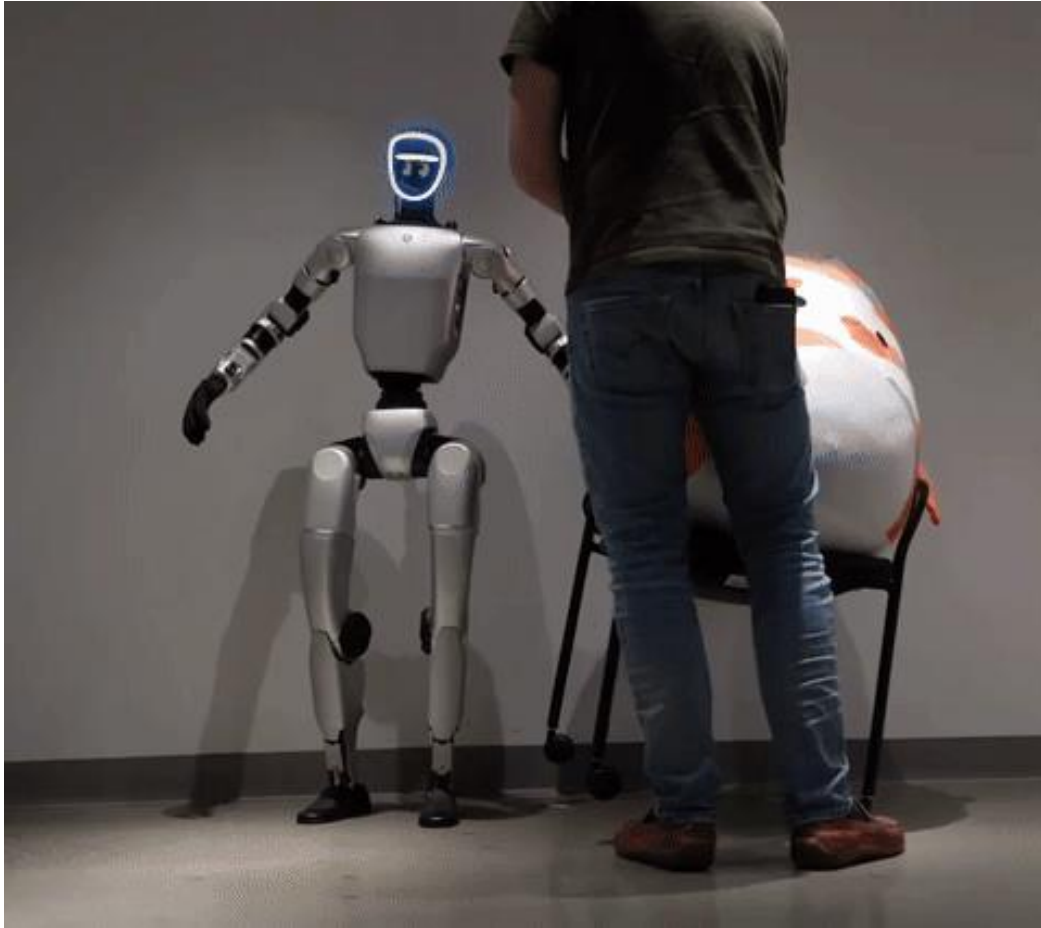
What Makes the Robot Behaves So Differently?



What Makes the Robot Behaves So Differently?



What Makes the Robot Behaves So Differently?



Content

- Open-loop control vs. close-loop control
- PID controllers
- Stability analysis of PID controllers
- Control in robotic manipulation
 - The effect of PD gain
 - Joint-space control vs. task-space control
 - Position control vs. force control
 - Indirect force control (impedance control)

Take Another Look at PID Feedback Controller

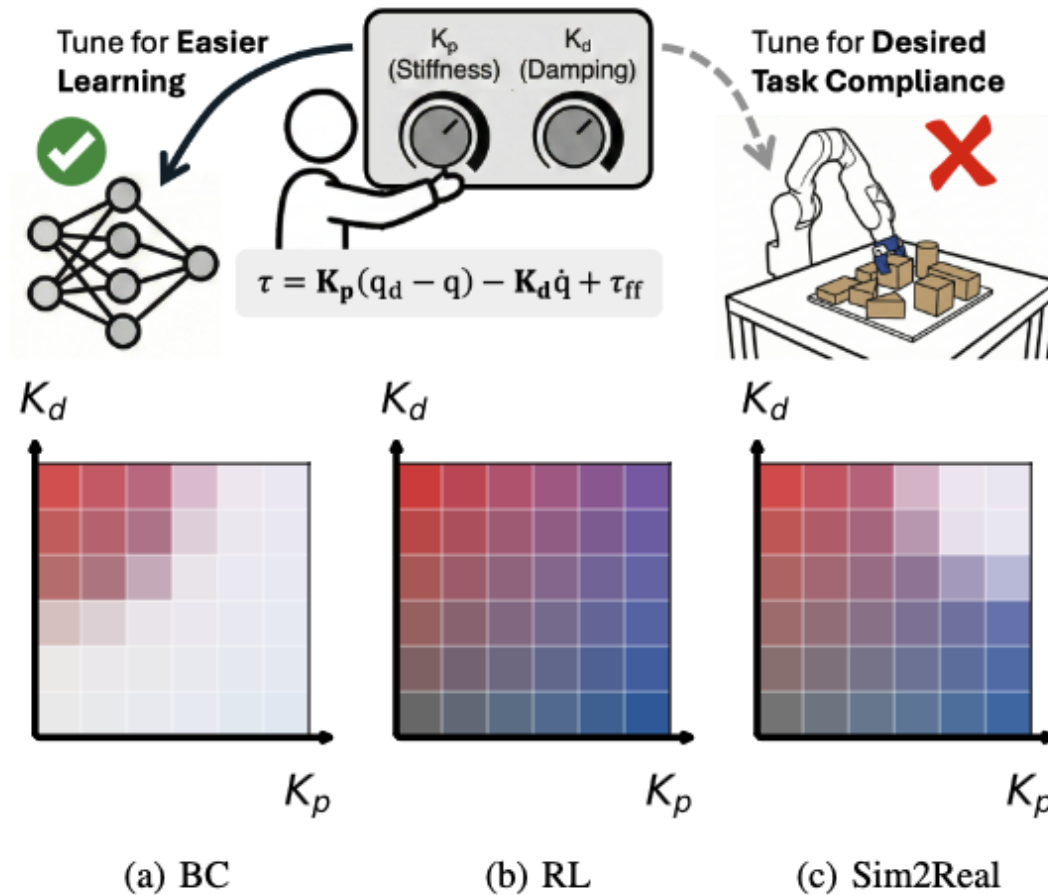
- PID feedback controller:
 - 1st-order error (immediate error)
 - Higher-order error (derivatives of errors)
 - Historical error (accumulated error)

$$u(t) = K_p q_e(t) + K_d \dot{q}_e(t) + K_i \int_0^t q_e(t) dt$$

How does the robot
behave with varying K_p ?

Demo Time

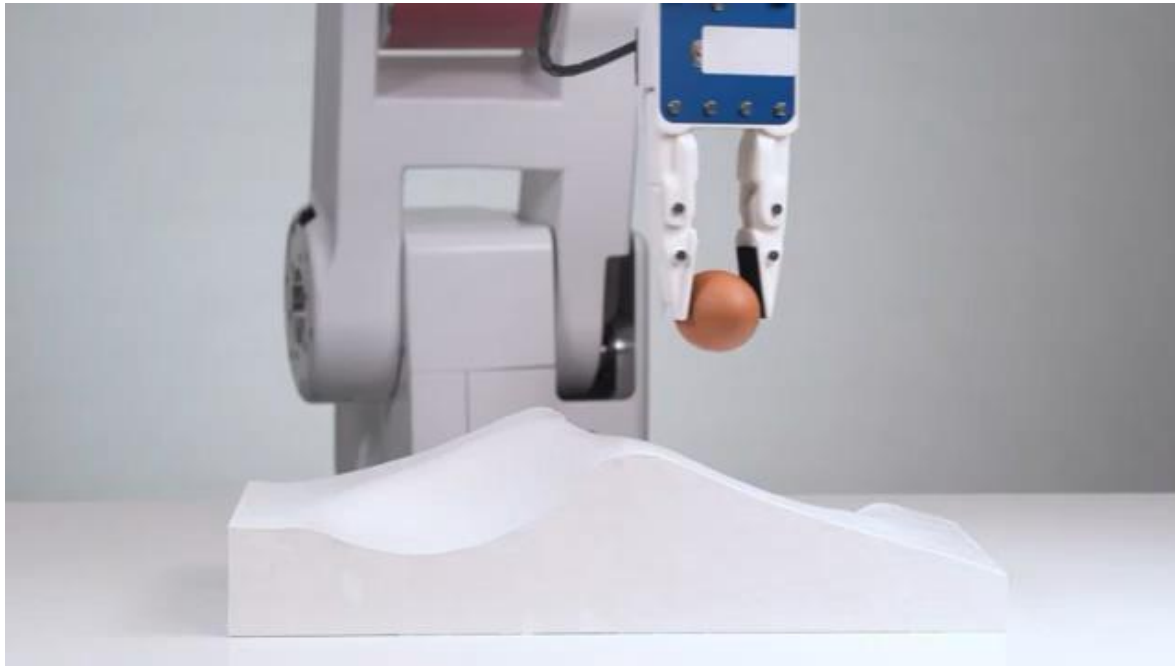
Different Robot Learning Approaches Prefer Different PID Controller Gains



Why Doesn't Low PID Gains Suffice Compliant Control?

- Turning down PID gains makes the robot look compliant, but doesn't allow it to behave compliantly
 - Energy accumulates with increasing positional errors
- PID controllers doesn't robot dynamics and external forces into consideration

Idea: Can We Control Desired Contact Force?



<https://www.youtube.com/watch?v=0ljgZ4Zt95U>



<https://www.youtube.com/watch?v=Oz1Aw0C2BeE>

Take Another Look at PID Feedback Controller

- PID feedback controller:

$$u(t) = K_p \boxed{q_e}(t) + K_d \dot{q}_e(t) + K_i \int_0^t q_e(t) dt$$

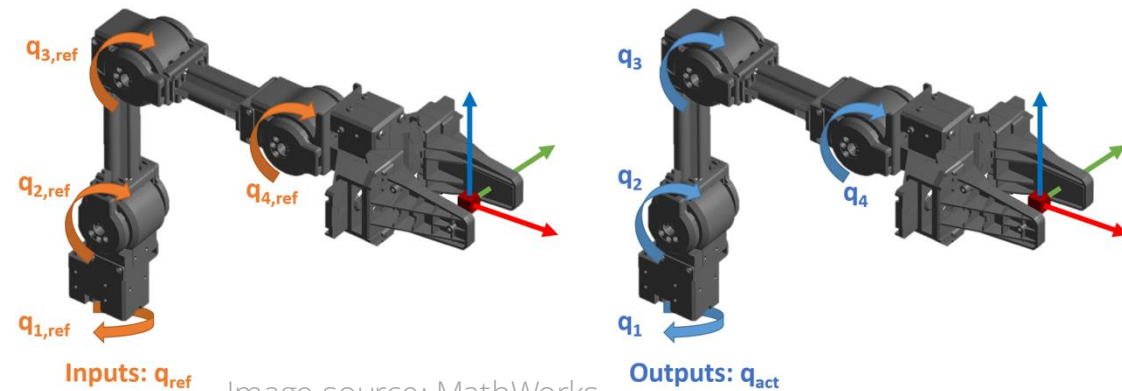
- The definition of motion error $\mathbf{q}_e = \mathbf{q}_{\text{desired}} - \mathbf{q}_{\text{current}}$
- The definition of motion $\mathbf{q}$?

Take Another Look at PID Feedback Controller

- PID feedback controller:

$$u(t) = K_p \boxed{q_e}(t) + K_d \dot{q}_e(t) + K_i \int_0^t q_e(t) dt$$

- The definition of motion error $\mathbf{q}_e = \mathbf{q}_{\text{desired}} - \mathbf{q}_{\text{current}}$
- The definition of motion $\mathbf{q}$?
- Because we often set $\mathbf{u}(t)$ as joint torques
 - $\mathbf{q}$ is often defined as joint angles / velocities / torques
- But desired contact force is better described by manipulators

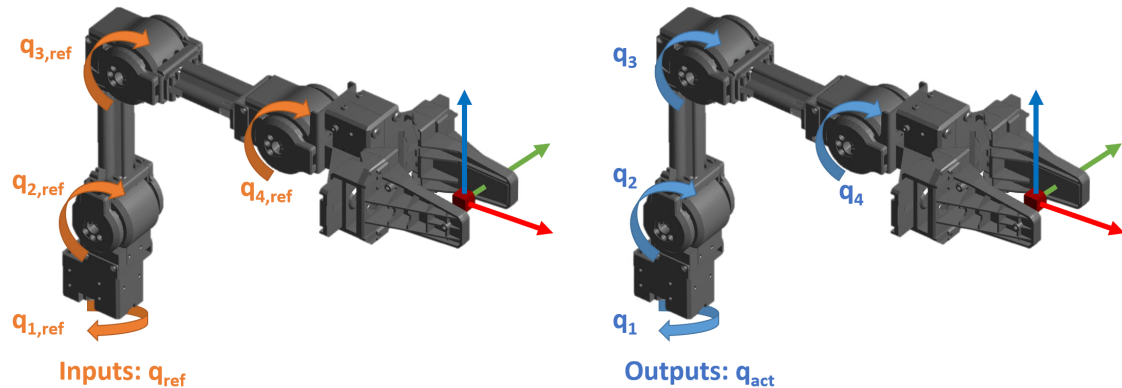


Content

- Open-loop control vs. close-loop control
- PID controllers
- Stability analysis of PID controllers
- Control in robotic manipulation
 - The effect of PD gain
 - Joint-space control vs. task-space control
 - Position control vs. force control
 - Indirect force control (impedance control)

Joint-space Control vs. Task-space Control

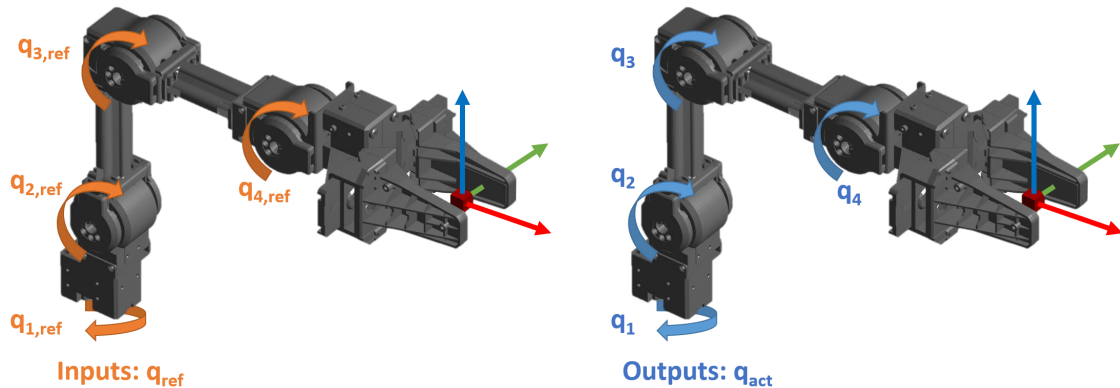
Joint-Space Control



$$\tau(t) = K_p q_e(t) + K_d \dot{q}_e(t) + K_i \int_0^t q_e(t) dt$$

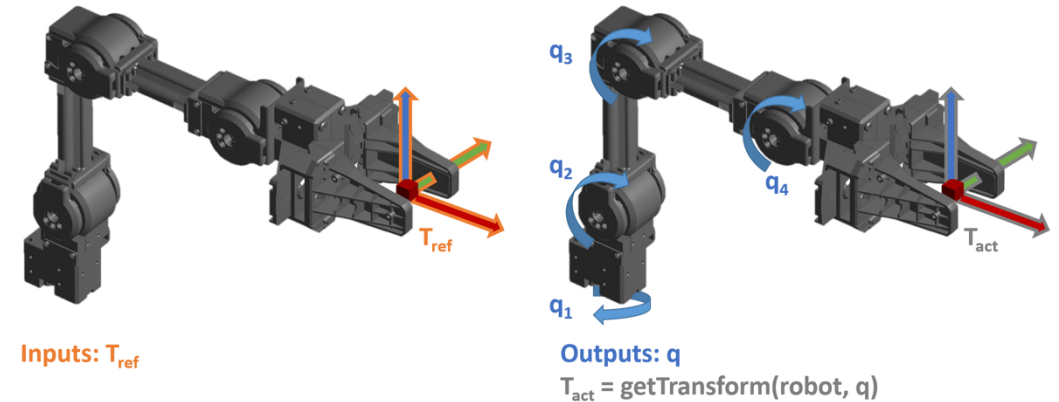
Joint-space Control vs. Task-space Control

Joint-Space Control



$$\tau(t) = K_p q_e(t) + K_d \dot{q}_e(t) + K_i \int_0^t q_e(t) dt$$

Task-Space Control



$$\tau(t) = \boxed{J^T(q(t))} \left(K_p X_e(t) + K_d \dot{X}_e(t) + K_i \int_0^t X_e(t) dt \right)$$

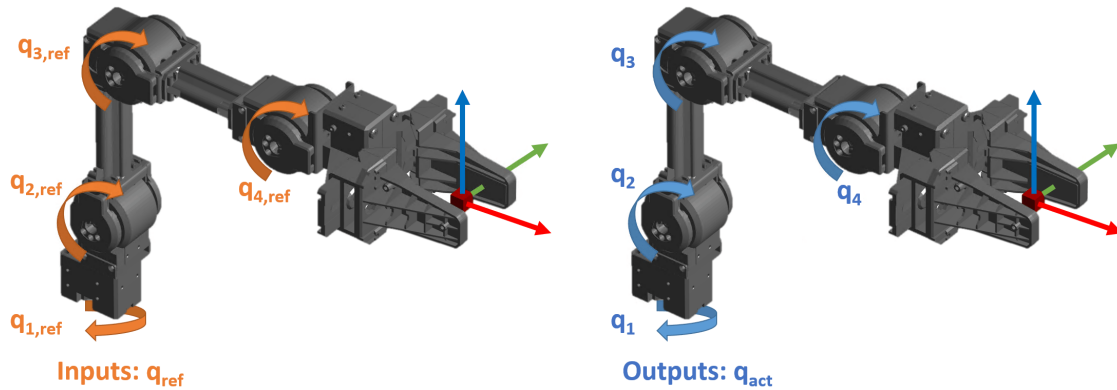
Contact Jacobian: transfer cartesian coordinates to joint-angle coordinates

Content

- Open-loop control vs. close-loop control
- PID controllers
- Stability analysis of PID controllers
- Control in robotic manipulation
 - The effect of PD gain
 - Joint-space control vs. task-space control
 - Position control vs. force control
 - Indirect force control (impedance control)

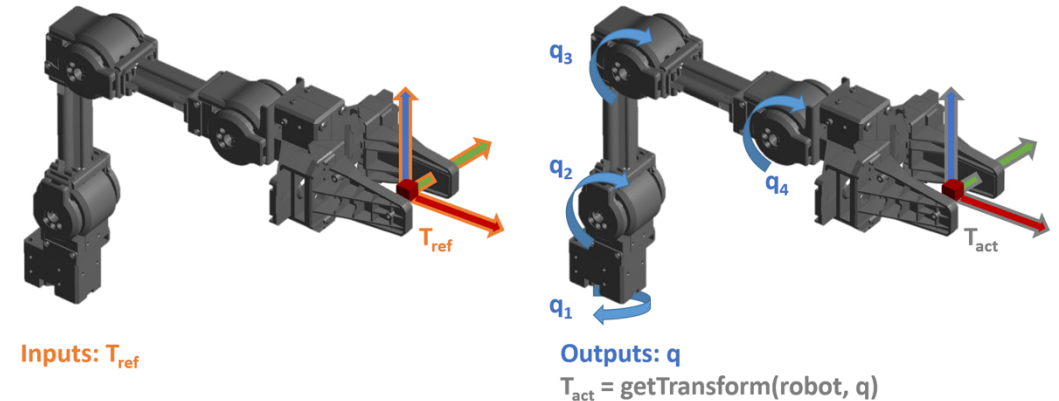
Control Desired Torque vs. Contact Force

Joint-Space Control



$$\tau(t) = K_p \tau_e(t) + K_d \dot{\tau}_e(t) + K_i \int_0^t \tau_e(t) dt$$

Task-Space Control



$$\tau(t) = \boxed{J^T(q(t))} \left(K_p F_e(t) + K_d \dot{F}_e(t) + K_i \int_0^t F_e(t) dt \right)$$

Contact Jacobian: transfer cartesian coordinates to joint-angle coordinates

Force Control Takes Dynamics into Consideration

- Manipulator dynamics

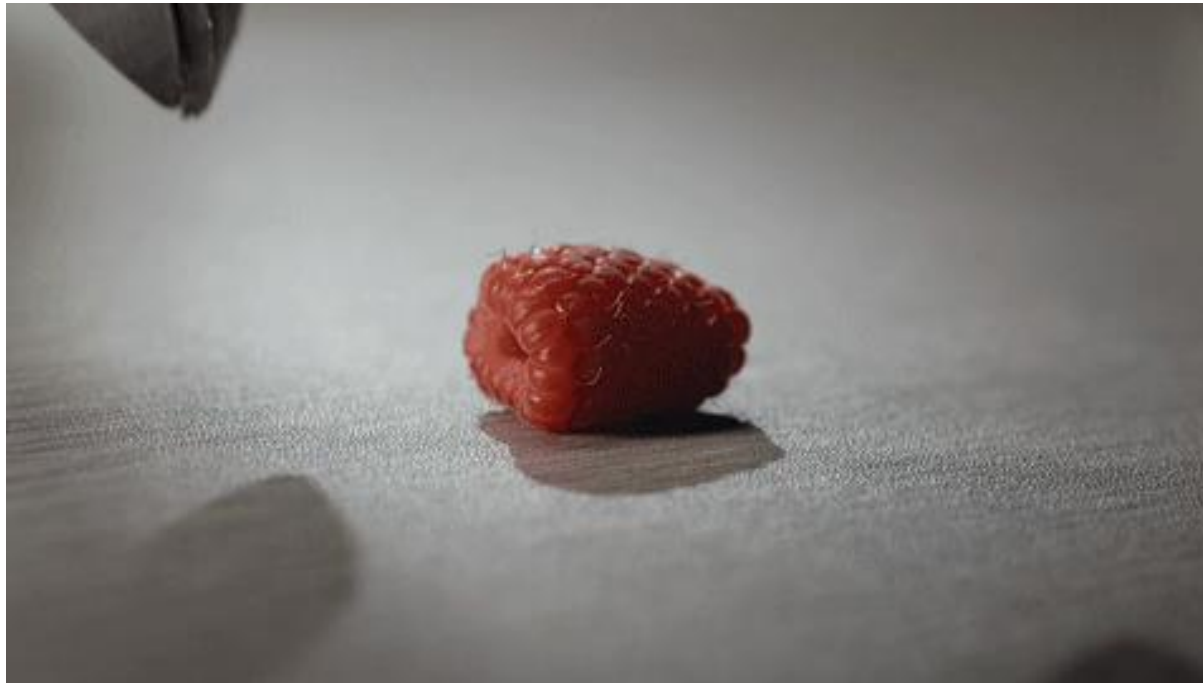
$$\tau = \underbrace{M(q)}_{\text{Mass}} \ddot{q} + \underbrace{c(q, \dot{q})}_{\text{Coriolis term}} + \underbrace{g(q)}_{\text{Gravity}} + J^T(q) F_{\text{tip}}$$

- PID Force controller with gravity compensation

$$\tau = \tilde{g}(q) + J^T(q) \left(F_d + K_p F_e(t) + K_d \dot{F}_e(t) + K_i \int_0^t F_e(t) dt \right)$$

Is Force Control All We Need?

- No!
- In most cases, we want the robot to track reference motion while being compliant



Video source: Eka Robotics

Hybrid Force-Position Control

- Manipulator dynamics

$$\tau = \underbrace{M(q)}_{\text{Mass}} \ddot{q} + \underbrace{c(q, \dot{q})}_{\text{Coriolis term}} + \underbrace{g(q)}_{\text{Gravity}} + J^T(q) F_{\text{tip}}$$

- Hybrid controller with gravity compensation

$$\tau = \tilde{g}(q) + J^T(q) \Lambda \left(K_p X_e(t) + K_d \dot{X}_e(t) + K_i \int_0^t X_e(t) dt \right) \text{Task-space motion control}$$

$$+ J^T(q) (1 - \Lambda) \left(F_d + K_p F_e(t) + K_d \dot{F}_e(t) + K_i \int_0^t F_e(t) dt \right) \text{Task-space force control}$$

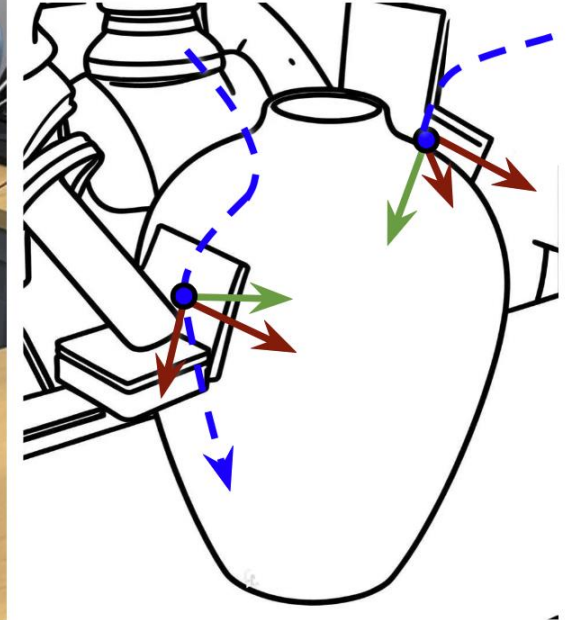
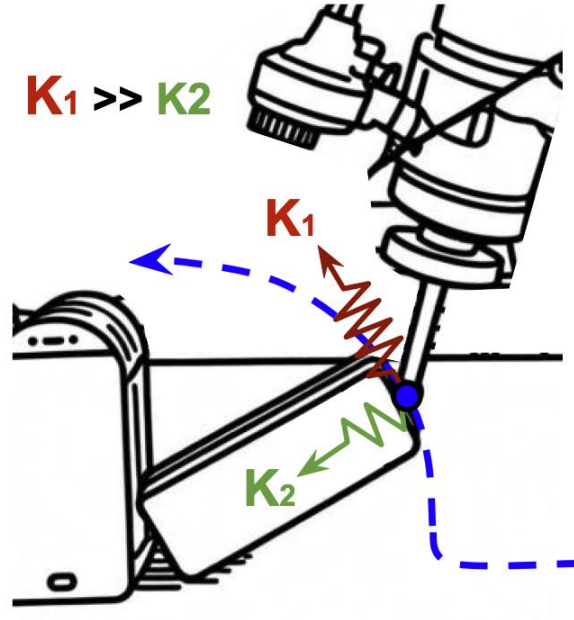
Content

- Open-loop control vs. close-loop control
- PID controllers
- Stability analysis of PID controllers
- Control in robotic manipulation
 - The effect of PD gain
 - Joint-space control vs. task-space control
 - Position control vs. force control
 - Indirect force control (impedance control)

How to Describe the Response at Hand-Object Contact?



How to Describe the Response at Hand-Object Contact?

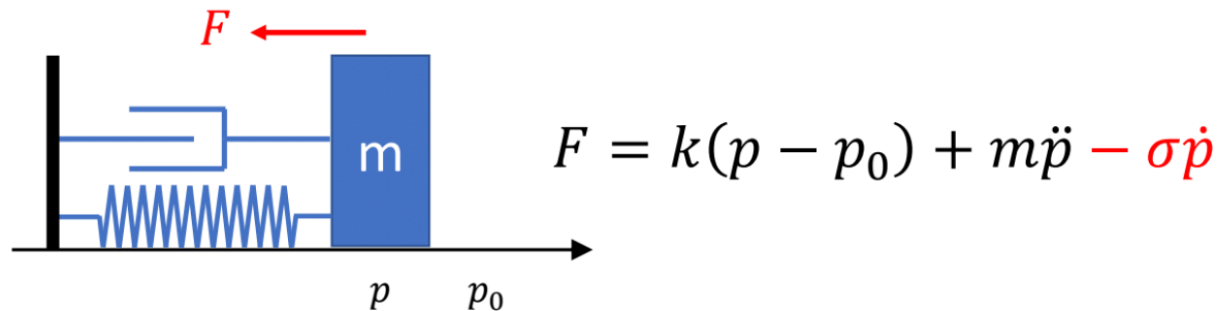
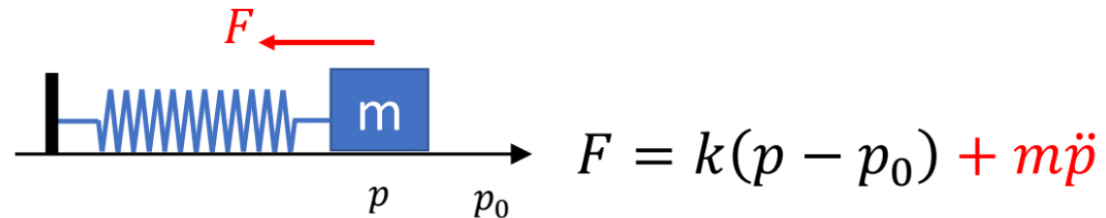
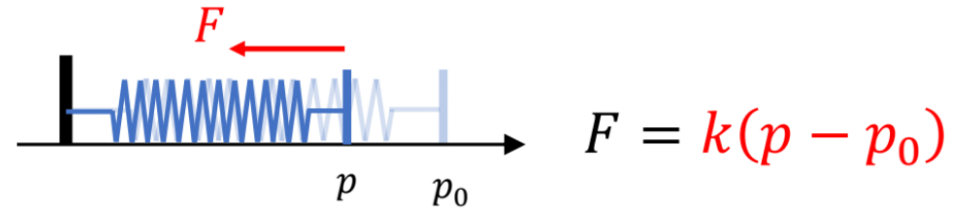


We can imagine as if a spring attached to each direction at the contact surface

Robot Behavior as Spring-Mass Systems

Describe Compliance

- Robot behaves like a:
 - Spring
 - Spring + Mass
 - Spring + Mass + Damper



Indirect Force Control

- We describe robot behaviors with spring-mass systems

$$-F_{\text{tip}} := M_d(\ddot{X} - \ddot{X}_d) + B_d(\dot{X} - \dot{X}_d) + K_d(X - X_d)$$

- Manipulator dynamics

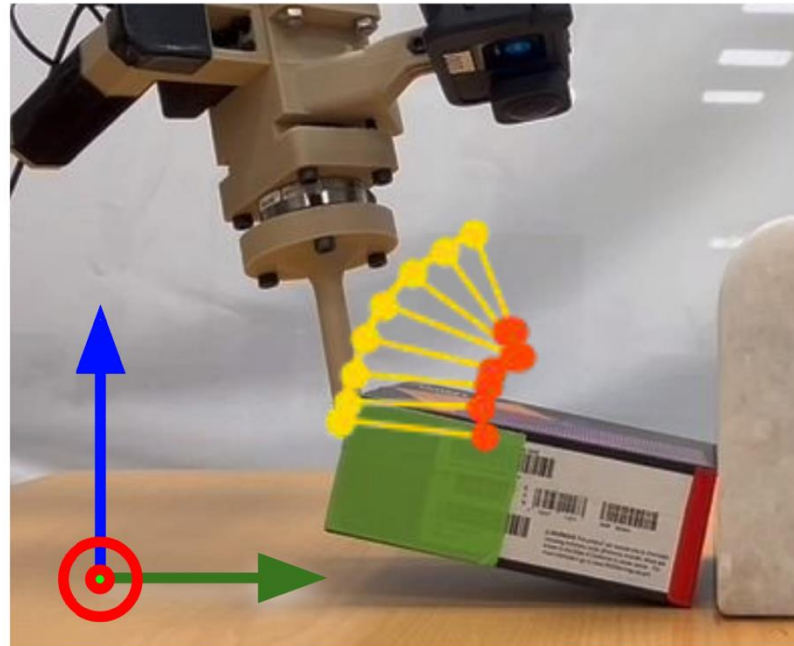
$$\tau = \underbrace{M(q)}_{\text{Mass}} \ddot{q} + \underbrace{c(q, \dot{q})}_{\text{Coriolis term}} + \underbrace{g(q)}_{\text{Gravity}} + J^T(q)F_{\text{tip}}$$

Impedance Control

- We describe robot behaviors with spring-mass systems

$$-F_{\text{tip}} := M_d(\ddot{X} - \ddot{X}_d) + B_d(\dot{X} - \dot{X}_d) + K_d(X - X_d)$$

How much force should the robot respond
w.r.t the current motion deviation



Impedance Control

- We describe robot behaviors with spring-mass systems

$$-F_{\text{tip}} := M_d(\ddot{X} - \ddot{X}_d) + B_d(\dot{X} - \dot{X}_d) + K_d(X - X_d)$$

How much force should the robot respond
w.r.t the current motion deviation

- Manipulator dynamics

$$\tau = \underbrace{M(q)}_{\text{Mass}} \ddot{q} + \underbrace{c(q, \dot{q})}_{\text{Coriolis term}} + \underbrace{g(q)}_{\text{Gravity}} + J^T(q)F_{\text{tip}}$$

- Impedance controller

$$\tau = \tilde{g}(q) - J^T(q) \left(M_d(\ddot{X} - \ddot{X}_d) + B_d(\dot{X} - \dot{X}_d) + K_d(X - X_d) \right)$$

Admittance Control

- We describe robot behaviors with spring-mass systems

$$M_d(\ddot{X} - \ddot{X}_d) + B_d(\dot{X} - \dot{X}_d) + K_d(X - X_d) := -F_{\text{tip}}$$

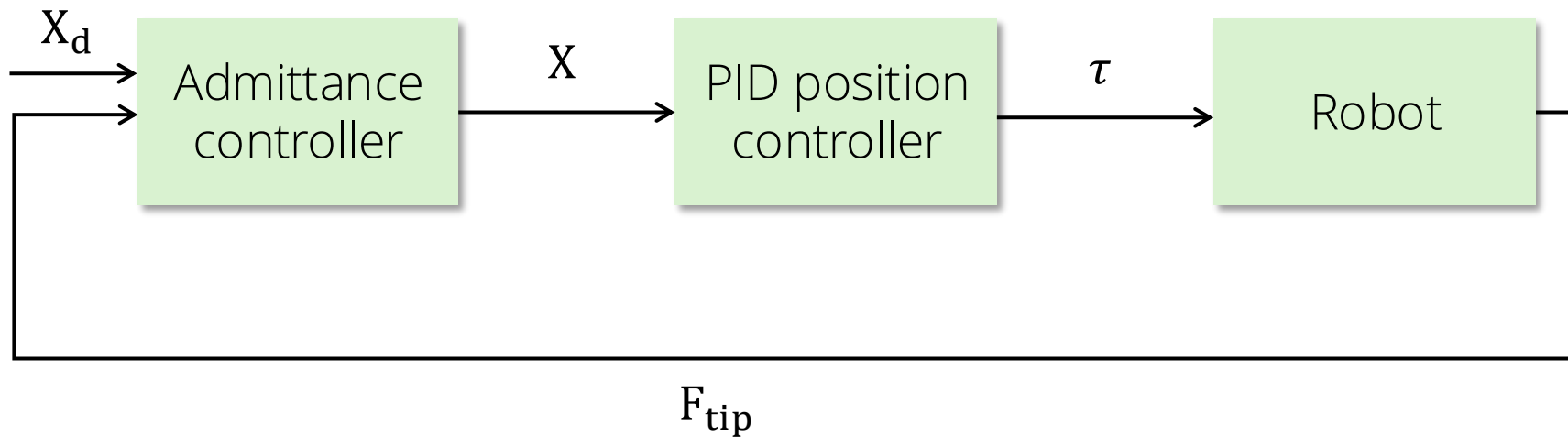
How should the robot move in response to
the current interacting force

Admittance Control

- We describe robot behaviors with spring-mass systems

$$M_d(\ddot{X} - \ddot{X}_d) + B_d(\dot{X} - \dot{X}_d) + K_d(X - X_d) := -F_{\text{tip}}$$

How should the robot move in response to the current interacting force





Impedance Control

Soft robot, but can be strong

Position deviation $\rightarrow$ force

Dominate characteristics: Spring

Pros:

Making contacts stably

Cons:

Requires accurate system id

Requires force inner loop

Admittance Control

Stiff robot, but can be delicate

Force $\rightarrow$ position change

Dominate characteristics: Inertia

Pros:

No need for system id

High maximum stiffness

Cons:

Bad at making contacts

Require force sensing



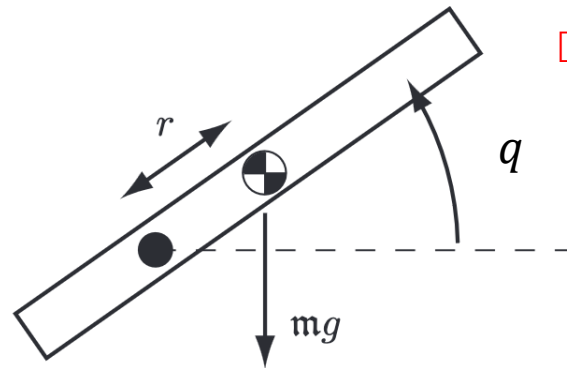
Appendix

How About Force Inputs?

- Velocity inputs are limited to applications with low/predictable force-torque requirements. Here, we consider control force inputs.
- Let's take a single-joint robot for example

$$\tau = M\ddot{q}(t) + mgr \cos q(t) + \underbrace{b\dot{q}(t)}$$

Damping (e.g friction)



- PID controller: $\tau = K_p q_e(t) + K_d \dot{q}_e(t) + K_i \int_0^t q_e(t) dt$

Proportional Derivatives Controller

- PD controller: $\tau = K_p q_e(t) + K_d \dot{q}_e(t)$
- Let's consider the robot is placed on a plane ($g = 0$)

$$M\ddot{q}(t) + mgr \cos q(t) + b\dot{q}(t) = K_p q_e(t) + K_d \dot{q}_e(t)$$

$$\Rightarrow M\ddot{q}(t) + b\dot{q}(t) = K_p(q_d(t) - q(t)) + K_d(\dot{q}_d(t) - \dot{q}(t))$$

Proportional Derivatives Controller

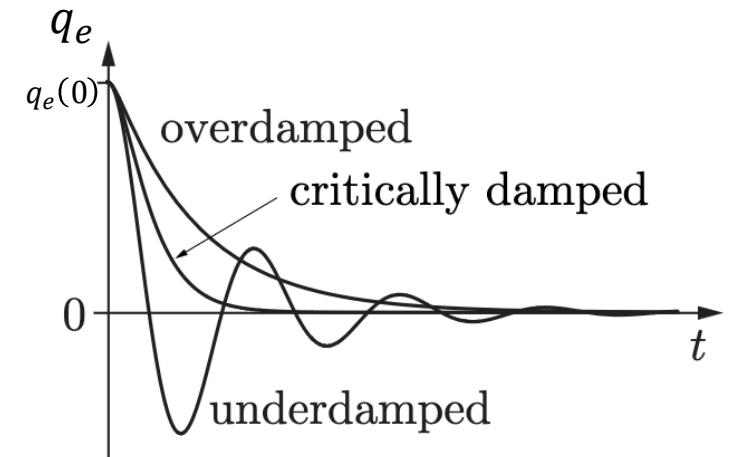
- PD controller: $\tau = K_p q_e(t) + K_d \dot{q}_e(t)$
- Let's consider the robot is placed on a plane ($g = 0$)

$$M\ddot{q}(t) + b\dot{q}(t) = K_p(q_d(t) - q(t)) + K_d(\dot{q}_d(t) - \dot{q}(t))$$

- Case 1: if $q_d(t) = c$ is constant
 - We have $\dot{q}_d = \ddot{q}_d = 0$, $q_e = c - q$, $\dot{q} = -\dot{q}_e$, $\ddot{q} = -\ddot{q}_e$
 - Substituting $q, \dot{q}, \ddot{q}$, we have

$$M\ddot{q}_e(t) + (b + K_d)\dot{q}_e(t) + K_p q_e(t) = 0$$

Doesn't this look familiar?



Proportional Derivatives Controller

- Let's consider the robot is placed vertically

$$M\ddot{q}(t) + mgr \cos q(t) + b\dot{q}(t) = K_p(q_d(t) - q(t)) + K_d(\dot{q}_d(t) - \dot{q}(t))$$

- Case 1: if $q_d(t) = c$ is constant

$$M\ddot{q}_e(t) + (b + K_d)\dot{q}_e(t) + K_p q_e(t) = mgr \cos q(t)$$

Very complex to get an explicit solution. But let's observe
when the system is stable

Proportional Derivatives Controller

- Let's consider the robot is placed vertically

$$M\ddot{q}(t) + mgr \cos q(t) + b\dot{q}(t) = K_p(q_d(t) - q(t)) + K_d(\dot{q}_d(t) - \dot{q}(t))$$

- Case 1: if $q_d(t) = c$ is constant

$$M\ddot{q}_e(t) + (b + K_d)\dot{q}_e(t) + K_p q_e(t) = mgr \cos q(t)$$


when the joint comes to rest ($\ddot{q}_e(t) = 0$ and $\dot{q}_e(t) = 0$),
the final error $q_e(t) \neq 0$:

$$K_p q_e(t) = mgr \cos q(t)$$

In other words, the robot has to provide a torque to hold
the link at rest $\theta \neq \pm \frac{\pi}{2}$, which happens when $q_e \neq 0$

Add Integral Controller to Reduce the Bias

- Let's consider the robot is placed vertically and a PID controller is used:

$$M\ddot{q}_e(t) + (b + K_d)\dot{q}_e(t) + K_p q_e(t) + K_i \int_0^t q_e(t) dt = mgr \cos q(t)$$

- Still a complex equation.... Let's envision what happens when the system is close to equilibrium $q_e = 0$ (and thus $\dot{q}$ is also close to 0), since the desired $q_d(t)$ is constant :

(Take derivatives)

$$\rightarrow M\ddot{q}_e(t) + (b + K_d)\ddot{q}_e(t) + K_p\dot{q}_e(t) + K_i q_e(t) = 0$$

PID Controller

$$M\ddot{q}_e(t) + (b + K_d)\dot{q}_e(t) + K_p q_e(t) + K_i q_e(t) = 0$$

- The characteristic equation is

$$s^3 + \frac{b + K_d}{M} s^2 + \frac{K_p}{M} s + \frac{K_i}{M} = 0$$

- The conditions for q_e to converge to zero

$$\begin{aligned} K_d &> -b \\ K_p &> 0 \\ \frac{(b + K_d)K_p}{M} &> K_i > 0. \end{aligned}$$

Require parameter tuning!

